DATA ENGINEERING MAJOR

END OF YEAR INTERNSHIP REPORT

Developing a Predictive Model for Cement Quality Assessment

Elaborated by:
CHERKAOUI NABIL

Supervised by:
Pr. EL FKIHI SANAA
Mr. KHALDI MOHAMED

- Academic Year : 2022/2023 -

ACKNOWLEDGMENT

Prior to any elaboration on the subject, it only seems fair to begin this report by addressing my gratitude to my supervisor for his mentorship and companionship throughout the project, and for doing his best to provide me with indispensable advices and resources during times of difficulties.

My gratitude touches on my dear parents for their patience, sacrifice and support along the way.

I also want to thank my professors at ENSIAS for their training and assistance, that which allowed me to develop the skills required to perform such work.

ABSTRACT

This report is a synthesis of the 2 months summer internship carried out within Admiral Digital Consulting.

The project aims to use ML algorithms and techniques in order to predict the cement quality for one of the numerous clients of Admiral Digital Consulting; Ciment de l'Atlas or CIMAT. After completing the preprocessing steps and evaluating different models, we ultimately employed the Random Forest algorithm, a type of Ensemble Learning method that falls within the category of bagging algorithms. Bagging involves training multiple instances of the same model on different subsets of the training data to reduce variance and enhance generalization.

Ensemble learning combines the predictions of multiple individual models to improve overall predictive accuracy and robustness.

After completing the model development phase, the deployment process marked the transition from concept to practical application. The developed predictive model was operationalized using a FastAPI API, which was deployed on the Railway platform. This deployment facilitated real-time interaction with the model, enabling seamless integration into various applications and workflows.

To evaluate the designed models, we used two regression performance criteria :Mean Squared Error or MSE and R2 score. As for the technological environment of the project, we essentially used Python Jupyter notebooks in VS code to code all parts of the project regarding the development phase and regular Python files regarding the deployment phase of our project, with libraries such as scikit-learn, pandas, FastAPI and more. We also carried out our project following the AutoML approach using Datarobot, a leading AutoML platform.

Key words: Machine Learning, Ensemble Learning, Bagging, Random Forest, Mean Squared Error, R2, Model Deployment, FastAPI, AutoML, Datarobot

RÉSUMÉ

Ce rapport est une synthèse du stage d'été de 2 mois effectué au sein d'Admiral Digital Consulting.

Le projet vise à utiliser des algorithmes et des techniques de ML afin de prédire la qualité du ciment pour l'un des nombreux clients d'Admiral Digital Consulting; Ciment de l'Atlas ou CIMAT. Après avoir effectué les étapes de prétraitement et évalué différents modèles, nous avons finalement utilisé Random Forest, un algorithme appartenant aux algorithmes de type Ensemble Learning et qui fait plus particulièrement partie de la catégorie des algorithmes de bagging. Le Bagging, consiste à former plusieurs instances du même modèle sur différents sous-ensembles de données d'apprentissage afin de réduire la variance et d'améliorer la généralisation.

L'Ensemble Learning combine les prédictions de plusieurs modèles individuels afin d'améliorer la précision et la robustesse des prédictions.

Après avoir achevé la phase de développement du modèle, le processus de déploiement a marqué la transition du concept à l'application pratique. Le modèle prédictif développé a été opérationnalisé à l'aide d'une API FastAPI, qui a été déployée sur la plateforme Railway. Ce déploiement a facilité l'interaction en temps réel avec le modèle, permettant une intégration transparente dans diverses applications et flux de travail.

Pour évaluer les modèles conçus, nous avons utilisé deux critères de performance de régression : l'erreur quadratique moyenne ou MSE en anglais et le score R2. En ce qui concerne l'environnement technologique du projet, nous avons essentiellement utilisé des notebooks Jupyter sur VS Code pour coder toutes les parties du projet concernant la phase de développement et des fichiers Python basiques concernant la phase de déploiement de notre projet, avec des bibliothèques telles que scikit-learn, pandas, FastAPI et autres. Nous avons également réalisé notre projet en suivant l'approche AutoML, en utilisant Datarobot, qui est une plateforme AutoML de premier plan.

Mots clés: Machine Learning, Ensemble Learning, Bagging, Random Forest, Erreur Quadratique Moyenne, R2, Déploiement du modèle, FastAPI, AutoML, Datarobot

GLOSSARY OF TERMS

Glossary of terms	
Abbreviation	**Meaning**
API	Application Programming Interface
AutoML	Automative Machine Learning
CIMAT	Ciment de l'Atlas
CSV	Comma Separated Values
KNN	K-Nearest Neighbors
LGBM	Light Gradient Boosting Machine
ML	Machine Learning
MSE	Mean Squared Error
Q-Q plot	Quantile-Quantile plot
SVR	Support Vector Regression
XGBoost	eXtreme Gradient Boosting

LIST OF FIGURES

CONTENTS

GENERAL INTRODUCTION

Morocco, a North African nation characterized by its vibrant culture and diverse landscapes, is also a pivotal player in the construction industry. Cement, a fundamental building block of urbanization, is a critical component in Morocco's development. As the country witnesses an upswing in construction activities, driven by infrastructure projects, urban expansion, and housing needs, the demand for cement escalates in tandem. However, this surge is accompanied by a fundamental challenge: the need for efficient raw material quality assessment to ensure the integrity of cement products. The conventional practice of subjecting raw materials to prolonged lab tests, lasting weeks or even months, not only disrupts production timelines but also strains financial resources. This is where our project comes into focus – we aimed to build predictive models that could anticipate the quality of raw materials. By preemptively determining the suitability of raw materials for the production line, these models would alleviate the burden of extended lab testing and resource allocation.

In our pursuit, we undertook two simultaneous approaches. The initial path involved a meticulous construct-from-scratch approach utilizing the Python programming language. This method demanded an intricate engagement with data preprocessing, algorithm selection, model training and evaluation. The resultant model was then seamlessly deployed using Python frameworks, actualizing its pragmatic use within real-world production scenarios.

At the same time, our second approach revolved around tapping into the capabilities of Automated Machine Learning (AutoML), an innovative methodology. This approach, renowned for its efficiency, expedites model development and deployment by automating key stages such as algorithm choice, hyperparameter tuning, and feature enhancement. By doing so, it not only accelerates the developmental trajectory but also alleviates the intricacies conventionally linked with manual model construction.

The fundamental motive behind this internship was the creation of robust predictive models that facilitate real-time evaluation of raw material quality in cement production. As raw materials undergo protracted laboratory tests, consuming extensive time and financial resources, the developed models emerge as game-changers.

To wrap up, this report presents a thorough investigation into predictive modeling, situated in the context of evaluating raw material quality in cement production. We offer a comprehensive view of the methodologies used, challenges faced, and achievements gained. We will now delve into the specifics of these methods, unveiling their unique characteristics and contributions to the field in upcoming sections.

CHAPTER 1

PROJECT CONTEXT

1.1 Introduction

This chapter will present the general context of the project by giving an overview of Admiral Digital Consulting where our end-of-year internship took place, the problematic and the objectives. Towards the end of this chapter, we will outline the planning of our project's development.

1.2 Admiral Digital Consulting Overview

Founded in 2014, Admiral Digital Consulting is a leading consulting firm and digital services integrator which aims to support organisations in their digital transition. Admiral facilitates enterprises in reshaping their operations through the assimilation of inventive remedies. In order to assure its clientele of top-notch assistance, Admiral has formed collaborative alliances with foremost technology providers in each specific domain. The fusion of adept consulting services and pre-packaged pioneering technological resolutions equips Admiral to deliver impartial guidance to clients and to implement tailored, streamlined solutions effectively.

Admiral Digital Consulting Missions

- Consulting and data auditing
- Predictive modeling
- Transformation to a data-centric organization
- Architecture, piloting and management of data projects
- Data Science

- Development of industrialization and integration of fast data, data science, organization and change management

1.3 Problematic

This project was realized in collaboration with the company Ciment de l'Atlas (CIMAT), a client of Admiral Digital Consulting, who shared with us the needed data to carry out this task.

CIMAT operates as follows when producing cement: the organization begins by ordering raw materials from a number of suppliers, which are then sent to the production line, ultimately producing the required cement. But the quality of these raw materials vary a lot since CIMAT is dealing with a lot of suppliers, so an intensive screening is required of these raw materials every time CIMAT receives a new batch, this screening is in the form of multiple tests and measurements carried out in a laboratory that can take up to a month to conclude, the results of these tests ultimately determine whether the respective raw material is suitable for being sent to the production line or not.

Isn't there a way to dispose of this month-long screening procedure that costs time and money and thus optimizing the cement fabrication process?

1.4 Project's Objective

The goal of our project is to create a ML model that can predict whether the raw material will be suitable for production or not. We will train our model on the data provided by CIMAT that contains all the necessary variables to make a decision and then we'll deploy the aforementioned model in a production environment so it can predict new unseen input data.

We will approach this same project from two different angles: the first one consisting of creating and deploying a model from scratch using Python and the mainstream libraries needed while the second one will make use of some AutoML platforms so that we can compare the different workflows, the advantages and disadvantages of both techniques

1.5 Project's Domain Overview

1.5.1 What is Data Science?

Data science is the field of applying advanced analytics techniques and scientific principles to extract valuable information from data for business decision-making, strategic planning and other uses. It's increasingly critical to businesses: The insights that data science generates help organizations increase operational efficiency, identify new business opportunities and improve marketing and sales programs, among other benefits. Ultimately, they can lead to competitive advantages over business rivals.

Data science incorporates various disciplines – for example, data engineering, data preparation, data mining, predictive analytics, Machine Learning and Data Visualization, as well as

statistics, mathematics and software programming.[1]

1.5.2 What is Machine Learning?

Machine learning (ML) is an umbrella term that refers to a broad range of algorithms that perform intelligent predictions based on a data set. These data sets are often large, perhaps consisting of millions of unique data points. Extending upon classical techniques in statistical modelling, recent progress in Machine Learning has attained what appears to be a human level of semantic understanding and information extraction, and sometimes the ability to detect abstract patterns with greater accuracy than human experts.

Modern Machine Learning has emerged as a powerful tool due to the vastly increased volumes of data, exponential growth in computational power and advances in algorithm design, driven by the needs of web industries.[2]

There are primarily three types of machine learning: Supervised, Unsupervised and Reinforcement Learning:

- **Supervised Learning:** Supervised learning is a type of machine learning that uses labeled data to train ML models. In labeled data, the output is already known.These algorithms take the labeled inputs and map them to the known outputs, which means the target variable is already known. Supervised Learning methods need external supervision to train the predictive models. Hence, the name supervised. They need guidance and additional information to return the desired result.
- **Unsupervised Learning:** Unsupervised Learning however, is a type of machine learning that uses unlabeled data to train machines. Unlabeled data doesn't have a fixed output variable. The model learns from the data, discovers the patterns and features in the data, and returns the output. The algorithms that fall under the scope of this type finds patterns and understands the trends in the data to discover the output. So, the model tries to label the data based on the features of the input data.
- **Reinforcement Learning:** Reinforcement Learning trains a machine to take suitable actions and maximize its rewards in a particular situation. It uses an agent and an environment to produce actions and rewards. The agent has a start and an end state. In this learning technique, there is no predefined target variable. Reinforcement learning follows trial and error methods to get the desired result, after accomplishing a task, the agent receives an award. An example could be to train a dog to catch the ball. If the dog learns to catch a ball, you give it a reward, such as a biscuit. Reinforcement Learning methods do not need any external supervision to train models.

1.5.3 Planning of Internship

1.5.3.1 Steps of a usual Data Science Project

In the realm of Data Science, a conventional project typically unfolds through a well-defined series of steps. The initial phase involves Data Collection, where pertinent data from diverse

sources is aggregated. This could encompass structured information stored in databases, as well as unstructured content like text and images. Data Preprocessing comes next, necessitating the refining, cleaning and organizing of collected data. Tasks such as managing missing values, rectifying outliers, and transforming variables transpire to ready the data for analysis. Subsequently, the Predictive Modeling phase takes center stage. Here, appropriate machine learning or statistical models are selected and applied to the preprocessed data. These models are then trained and evaluated using a plethora of metrics. Once an effective model is discerned, the project advances to Model Deployment, this stage orchestrates the integration of the model into a production environment, enabling it to generate predictions for new, unseen data. It's important to recognize that data science, being an iterative process, may involve revisiting prior steps for refining insights or adjusting approaches based on emerging revelations.

1.5.3.2 Steps of our Internship Project

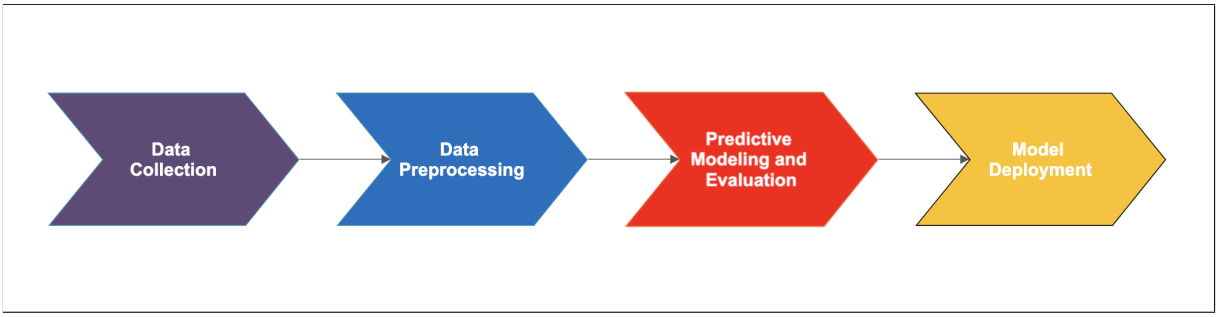


Figure 1.1: Usual Data Science project steps

1. **Data Collection:** We didn't perform any data collection technique since CIMAT had provided us already with a ready-to-use CSV document in which we applied the necessary data preprocessing techniques.
2. **Data Preprocessing:** After reading and exploring the data provided, we found that there was a considerable amount of missing values to be handled by using some imputation methods, there was also the need to perform a normalization of the data as the scale of the features was highly varying.
3. **Predictive Modelling and Evaluation:** We used the main stream regression ML algorithms after cleaning the data and using it to train the aforementioned algorithms, we trained all sorts of models from Linear Regression up to XGBoost and picked the one with the better performance so it can be deployed and used to predict new unseen data.
4. **Model Deployment:** After developing the model, we developed an API using FastAPI that accepts new raw data as input. This data undergoes the same preprocessing steps employed during the training phase before being fed into the model, the API then gives as an output the model's prediction. Finally, we deployed the API on Railway, a cloud hosting service.

1.6 Conclusion

In this first chapter, we began by presenting the organization where our end-of-year internship took place. Then, we defined the problematic, the main objectives and the planning in order to have a clear vision of the project.

CHAPTER 2

DATA

2.1 Introduction

In this chapter we present the data that was given to us by CIMAT, we'll perform on this data, after some exploration, two main preprocessing steps: value imputation and scaling. The goal here is to get a clean data that can be fed into a ML algorithm.

2.2 Data Collection

The process of data collection took on a distinct form due to the circumstances. Rather than engaging in the traditional data collection methods, we were presented with a preexisting and readily available CSV file by CIMAT. This file encompassed the essential dataset required for the project, rendering the need for independent data collection methods unnecessary. By leveraging this dataset, we could transition to the subsequent phases of the project, including data preprocessing, modeling, evaluating and optimizing the performances and finally deploying the model in a real world setting.

2.3 Data Exploration and Preprocessing

2.3.1 Data Exploration

We begin by opening the dataset in a Python notebook with the Pandas library, we see that the dataset is constituted by 12 features: the date column and the P1 to P11 columns, and the target column named "Objectif".

	Date	P1	P2	P3	P4	P5	P6	P7	P8	P9	P10	P11	Objectif
0	2020-01-01	99.098306	2.340254	1.713615	1.370000	1.075000	63.633100	NaN	NaN	0.715	6.140000	0.069838	47.966667
1	2020-01-02	98.722801	2.279820	1.608611	1.453333	0.986667	63.124600	2.745000	9.180	0.715	6.442500	0.069994	48.733333
2	2020-01-03	99.102621	2.339768	1.723449	1.443333	1.293333	62.416600	2.730000	9.405	0.715	6.459091	0.069090	NaN
3	2020-01-04	98.727026	2.336018	1.690402	1.470000	0.955000	63.241550	2.702222	8.878	0.715	6.370000	0.070069	NaN
4	2020-01-05	99.303335	2.350911	1.752401	1.476667	1.373333	62.584333	2.710000	9.120	0.715	6.145000	0.069843	NaN
...	...	...	...	...	...	...	...	...	...	...	...	...	...
379	NaT	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	0.725	NaN	0.070154	NaN
380	NaT	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	0.070412	NaN
381	NaT	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	0.070177	NaN
382	NaT	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	0.070094	NaN
383	NaT	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN

Figure 2.1: Raw data given by CIMAT

```
['Date', 'P1', 'P2', 'P3', 'P4', 'P5', 'P6', 'P7', 'P8', 'P9', 'P10', 'P11', 'Objectif ']
```

Figure 2.2: Features and the target column (Objectif) of our dataset

The dataset provides us with the outcomes of tests conducted on a sample of a raw material. The dataset consists of 384 entries along with 13 columns, encompassing 12 distinct features. Among these features, one column captures the test date named 'Date', while the remaining columns—designated as P1 through P11—represent readings acquired from sensors employed during the testing process. The final column in the dataset serves as the target variable, named 'Objectif'. This output column yields a numerical value, which signifies the test result. This outcome holds significance as it determines, in relation to a predefined threshold, whether the given raw material is deemed suitable for production purposes.

```
Number of Null values in target column: 256 out of 384 values
Number of non-Null values in target column: 128 out of 384 values
66.67% of target values are Null

Number of Null values in Date column: 7
Number of Null values in P1 column: 43
Number of Null values in P2 column: 43
Number of Null values in P3 column: 43
Number of Null values in P4 column: 42
Number of Null values in P5 column: 43
Number of Null values in P6 column: 43
Number of Null values in P7 column: 112
Number of Null values in P8 column: 116
Number of Null values in P9 column: 0
Number of Null values in P10 column: 93
Number of Null values in P11 column: 83
```

Figure 2.3: Null values for each column

Here we can see that a substantial amount of data is null, especially the target column in which more than half of it is empty, these null values along with the limited number of entries (384) posed a lot of challenges in order for us to build a good enough predictive model, but sadly the confidential policies of CIMAT prevented us from getting more quality data, so we focused instead on this internship in getting right the steps of our workflow with meaningful preprocessing techniques and we'll try to optimize to the most of our ability the performances of our model even though we don't expect it to be very useful in a real life environment since we have so little amount of training data.

2.3.2 Data Preprocessing

After identifying the presence of null values in both the feature and target columns, we proceed to perform data imputation for both. Concerning the target column we'll go with a model based imputation approach; we'll predict the missing targets with a ML model that will be trained on the existing target values, as for the features, we'll use both the mean imputation and the KNN imputation, we'll then choose which one will help us get better performances.

2.3.2.1 Model Based Imputation for the target column

We will divide the dataset into two subsets, one with only existing target values (33% of the overall data or 128 entries) and the other with only null target values, we'll train a model in the first subset and then predict the null values in the second subset with the aforementioned

model. But, before training a ML model on this data we have to handle the missing values in the features column that exist in this subset.

We'll compare between two imputation methods:

- **Mean Imputation:** The Mean imputation is a substitution method in which the missing values of a certain variable or feature are replaced by the mean of non-missing cases of that same variable or feature. This method can only be considered if the features follow a roughly normal distribution and if there is no significant skewness or outliers in the feature columns.
- **K-Nearest Neighbors Imputation:** This approach utilizes the K-Nearest Neighbors (KNN) method to replace the missing values in the dataset with the mean value from the parameter 'K' nearest neighbors found in the training set (in this project we'll use K = 5). By default, it uses an Euclidean distance metric to impute the missing values, so we'll replace any missing value with the mean of the 5 nearest non missing values. This method can be considered if the features follow non normal or complex distribution, event though it can also be used with a normal distribution, also outliers won't affect this imputer like it would the Mean one.

In order to use the Mean Imputation method, the feature column that will be imputed must follow a Normal distribution, we therefore need to check the distribution of our features.

Feature distribution: We need to check whether the feature columns all follow a normal distribution or not, there are two ways in order to determine that:

- **Visual Inspection: Histograms and Q-Q plots:** Visual methods, such as histograms and Q-Q plots, are valuable tools to assess the normality of a distribution. Histograms offer a visual representation of data distribution by depicting the frequency of values within specified intervals. In a normal distribution, the histogram typically exhibits a symmetrical bell-shaped curve, with data points clustered around the mean. Deviations from this pattern can suggest departures from normality. On the other hand, Q-Q plots provide a direct comparison between the quantiles of the dataset and the expected quantiles of a theoretical normal distribution. In an ideal normal distribution, the points on the Q-Q plot would closely follow a straight 45-degree reference line. Departures from this line, especially towards the tails, indicate differences from normality. Outliers or systematic deviations from the straight line can point to skewness, heavy tails, or other deviations.

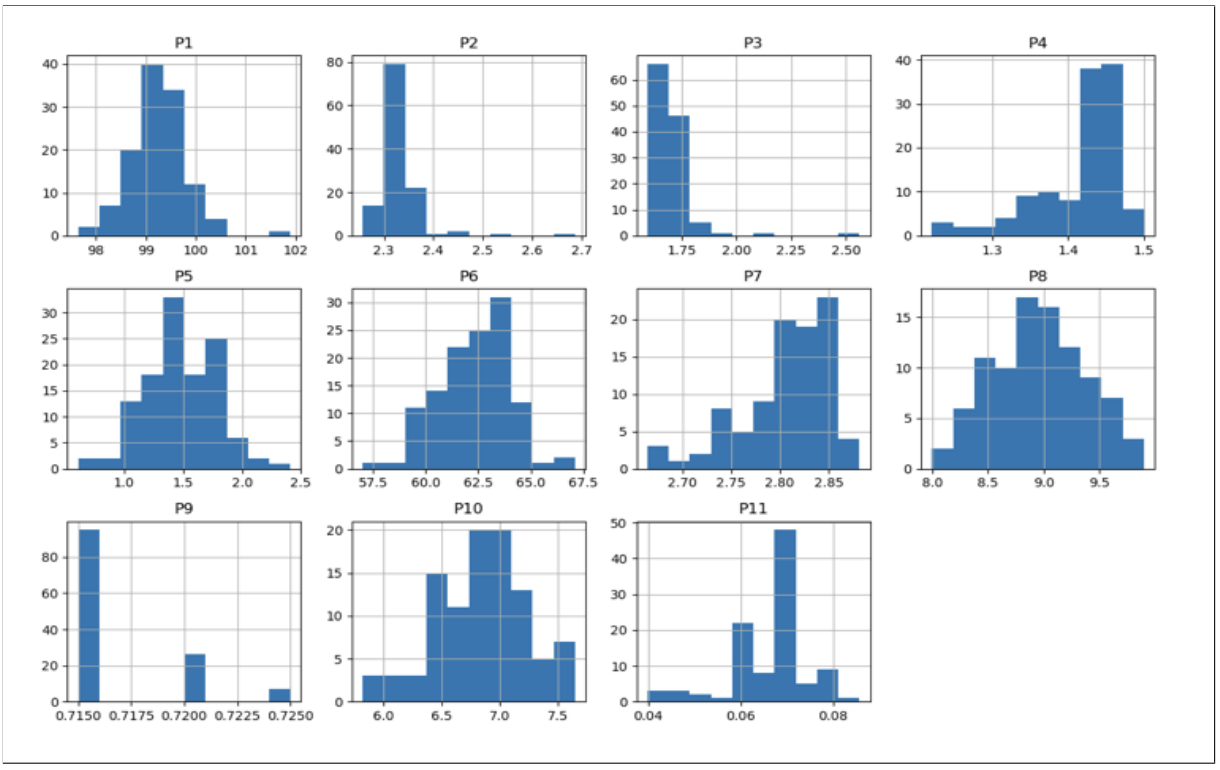


Figure 2.4: Histograms for each feature

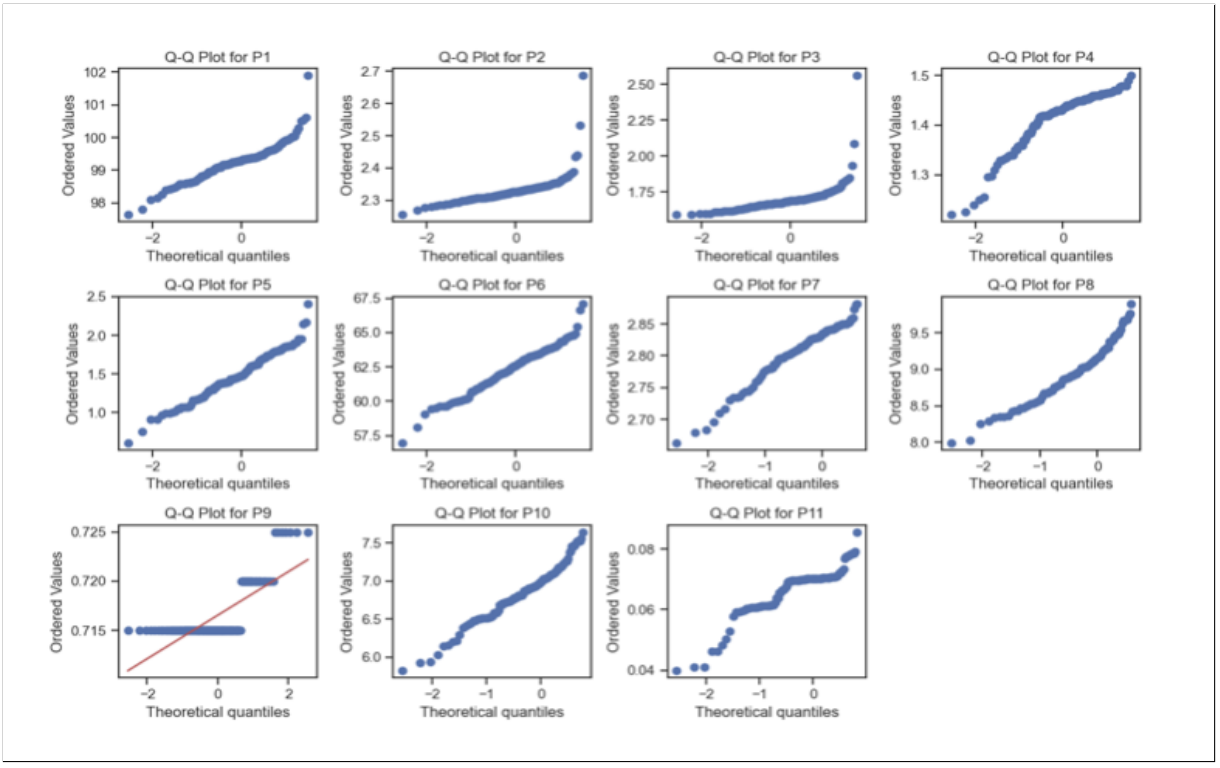


Figure 2.5: Q-Q plots for each feature

- **Statistical test: Shapiro-Wilk:** While the visual methods are insightful for initial assessments, it's important to remember that no dataset is perfectly normal, this is why we have to combine them with statistical tests so we can provide a more comprehensive understanding of distribution's normality. Statistical tests, such as the Shapiro-Wilk test, play a crucial role in assessing the normality of data distributions. The Shapiro-Wilk test evaluates whether a dataset follows a normal distribution by calculating a test statistic based on the differences between observed values and the values expected under a normal distribution. If the p-value associated with the test is below a certain significance level (often 0.05), it suggests that the data significantly deviates from normality.

```
Shapiro-Wilk test p-value for P1: 1.0
Shapiro-Wilk test p-value for P2: 1.0
Shapiro-Wilk test p-value for P3: 1.0
Shapiro-Wilk test p-value for P4: 1.0
Shapiro-Wilk test p-value for P5: 1.0
Shapiro-Wilk test p-value for P6: 1.0
Shapiro-Wilk test p-value for P7: 1.0
Shapiro-Wilk test p-value for P8: 1.0
Shapiro-Wilk test p-value for P9: 1.615270489154147e-17
Shapiro-Wilk test p-value for P10: 1.0
Shapiro-Wilk test p-value for P11: 1.0
```

Figure 2.6: The Shapiro-Wilk test performed on each feature

Now, we can conclude that all the column features follow a Normal distribution, except the feature P9, but we find that it does not have any missing value so it won't be impacted by the Mean Imputer, which can thus be used safely.

```
Shapiro-Wilk test p-value for P9: 1.615270489154147e-17
Number of null values in the P9 column is: 0
```

Figure 2.7: Shapiro-Wilk test and number of null values in the P9 column

Data Normalization: After performing both imputations, we'll plot the scatter plots for each features relative to the target, so we can get familiar with the imputed data.

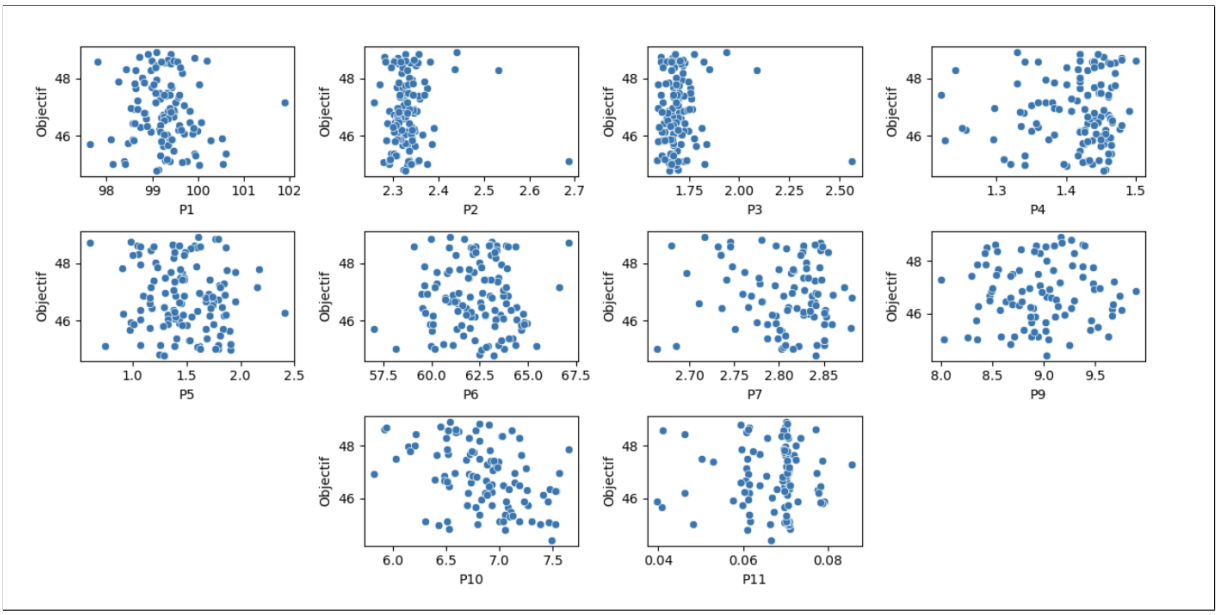


Figure 2.8: Feature distribution after Mean Imputation

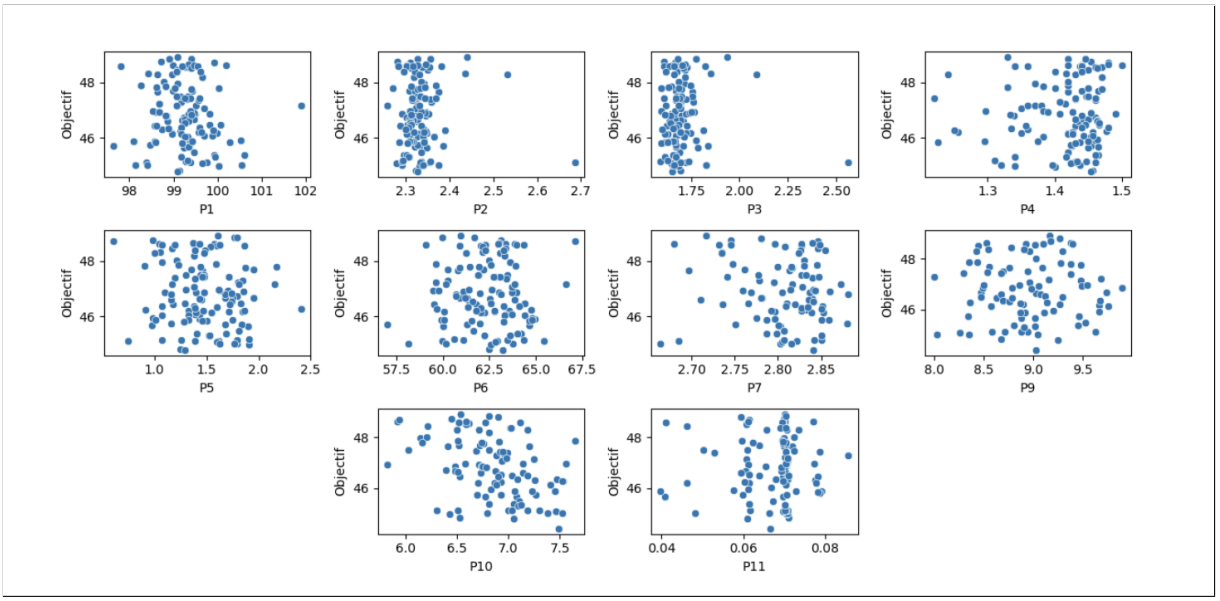


Figure 2.9: Feature distribution after KNN Imputation

When we examined how our features connect with the target variable, we noticed something interesting. The scales of these features vary quite a bit, which might impact our analysis. To address this, we need to normalize our data. This should help balance things out and ensure that each feature has an equal say when we dive into modeling.

- **Min-Max Scaler:** For the normalization we'll use the MinMax method that can be found in the sklearn library, the Min-Max normalization technique follows this transformation:

$$x' = \frac{x - \min(x)}{\max(x) - \min(x)}$$

Figure 2.10: Min-Max normalization where x is the original values and x' is the normalized value

Model creation on the first subset: Now after performing the preprocessing steps on the first subset, all missing values have been handled and all the data points are now between 0 and 1 as a result of normalization, we omitted the date column for now as it is not important in this part of the study.

	P1	P2	P3	P4	P5	P6	P7	P8	P9	P10	P11	Objectif
0	0.342087	0.194364	0.124242	0.535714	0.258333	0.656561	0.456355	0.452756	0.0	0.175342	0.655154	47.966667
1	0.253849	0.053583	0.015763	0.833333	0.209259	0.606283	0.372129	0.622047	0.0	0.341096	0.658557	48.733333
2	0.316515	0.162196	0.106658	1.000000	0.244444	0.631440	0.073507	0.452756	0.0	0.056621	0.661814	48.616667
3	0.234644	0.272241	0.170162	0.714286	0.472222	0.377002	0.148684	0.400700	0.0	0.756164	0.700510	47.666667
4	0.341008	0.427733	0.354395	0.392857	0.555556	0.392432	0.243492	0.616798	0.0	0.391781	0.659807	48.900000
...	...	...	...	...	...	...	...	...	...	...	...	...
123	0.277658	0.171437	0.053523	0.422619	0.305556	0.603952	1.000000	0.517060	0.0	0.529680	0.641350	46.816667
124	0.407261	0.125239	0.002190	0.273810	0.557407	0.556749	0.690440	0.813648	0.0	0.953973	0.665627	46.966667
125	0.335629	0.170292	0.097855	0.500000	0.311111	0.680529	0.961715	0.611549	0.0	0.637443	0.676233	47.166667
126	0.412821	0.127499	0.023709	0.404762	0.625926	0.495129	0.609495	0.606299	0.0	0.753425	0.504097	46.853333
127	0.529609	0.307047	0.227260	0.107143	1.000000	0.263813	0.854518	0.191601	0.0	0.933333	0.652283	46.266667

Figure 2.11: Our subset after performing imputation and normalization

We plot the scatter again for each imputation after normalization, and now we can notice that all the feature have been scaled, and we no longer have a scale disparity, we can now fit a model on this data.

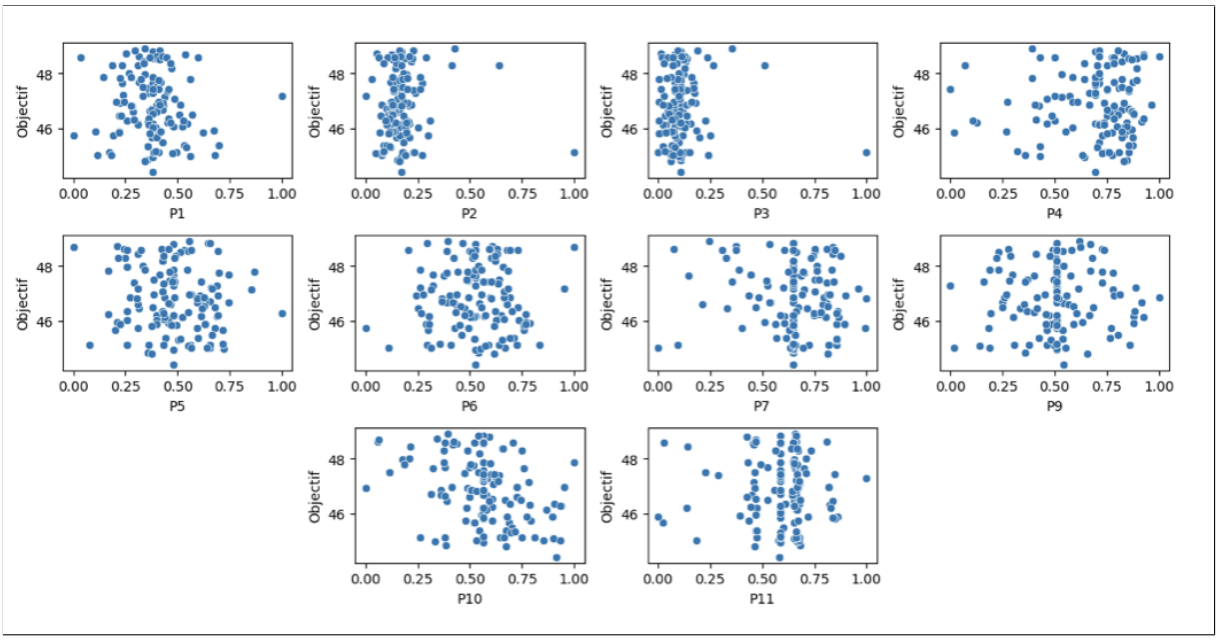


Figure 2.12: Feature distribution after Mean Imputation and after normalization

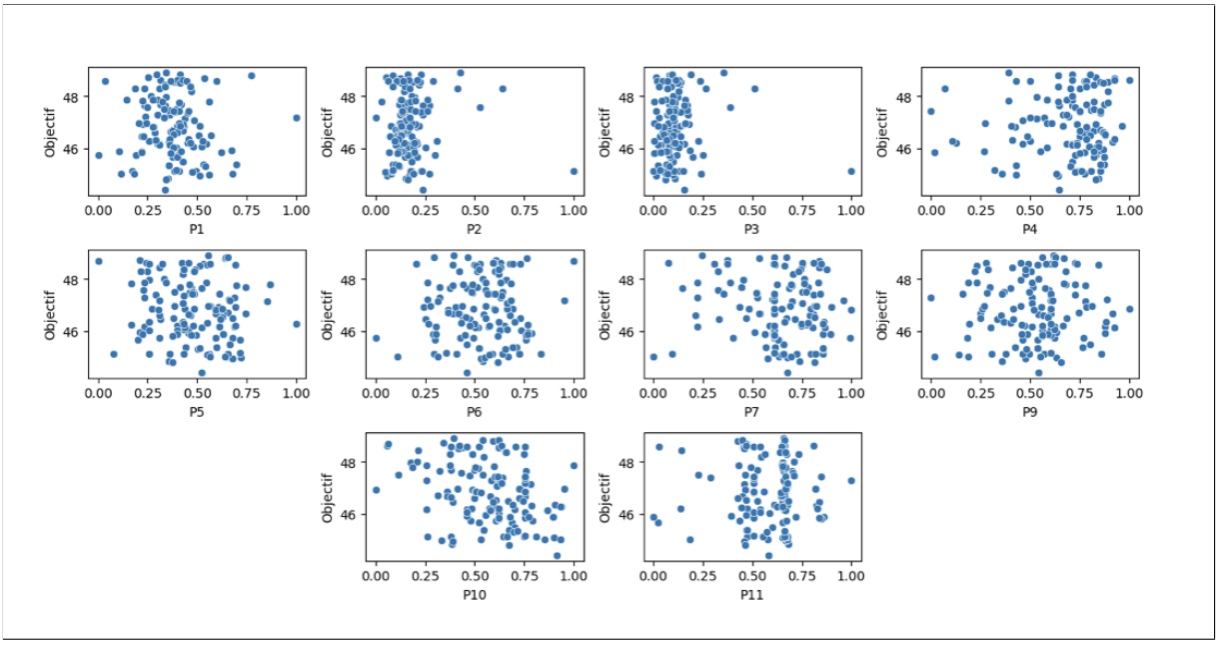


Figure 2.13: Feature distribution after KNN Imputation and after normalization

Light Gradient Boosting Machine (LGBM): LightGBM, or Light Gradient Boosting Machine, is an advanced machine learning technique that's particularly skilled at tackling complex problems, it is a free and open-source distributed gradient-boosting framework for machine learning, originally developed by Microsoft.

Through a series of steps, it gradually builds a robust model by combining the expertise of many simpler models (Decision Trees mainly), and is used for ranking, classification and other machine learning tasks. The development focus is on performance and scalability.

This is the model that we'll use on this first subset in order to predict the missing target values.

Combining the subsets: After fitting and training the LGBM model, we use it to predict the missing target values in the second subset, the same subset that we'll combine back with the first one after it went through the same preprocessing steps we saw earlier (Data Imputation and Normalization).

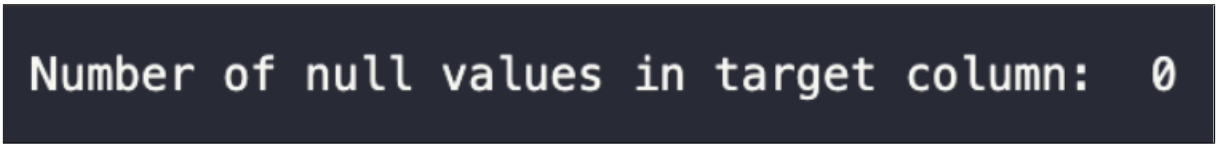


Figure 2.14: We no longer have any null value in the target column

2.3.2.2 Date column

Before creating and training a model on the original dataset we need to handle one last feature which is the Date feature. The Date feature is a categorical feature and is represented like so: the date of 22nd March 2020 is saved as 2020-03-22 in the dataset, and we need to convert it to an integer in order to be accepted as input by the model.

Cyclical Encoding: Cyclical encoding is a data transformation technique that plays a pivotal role in capturing cyclic or periodic patterns within features, such as dates, times, or angles. It operates by employing the sine and cosine functions to map feature values onto a unit circle, leveraging the full range of [-1, 1]. This ensures that cyclic features, like dates ranging from month to month or days within a month, are accurately represented. The two functions, sine and cosine, work in tandem to provide orthogonal encoding, meaning they offer complementary information about the cyclic pattern's position and elevation. In our context, this approach proves particularly valuable for encoding date features. It allows us to encapsulate temporal cyclic patterns, such as seasonal or monthly variations, ensuring that our machine learning models can effectively grasp and leverage these patterns for more robust and accurate predictions, all while maintaining a continuous and interpretable representation of time.

Here's a breakdown of how this encoding works:

- **Mapping to a Circle:** Each data point is mapped onto a unit circle where the circumference represents a full cycle of the periodic data (e.g., 0 to 360 degrees or 0 to 2π radians).
- **Sine and Cosine Transformation:** For each data point, cyclical encoding applies both the sine and cosine functions to map the point's position on the circle to two continuous

values. The sine function captures the vertical position (elevation) of the point, and the cosine function captures the horizontal position (position around the circle).

- **Orthogonal Encoding:** The use of both sine and cosine functions provides orthogonal encoding, which means that the two resulting values are uncorrelated. This encoding captures not only the cyclical position but also the phase or timing of the data within the cycle.
- **Periodicity:** By using trigonometric functions, cyclical encoding preserves the periodicity of the data. As the data cycles through its range, the encoded values will oscillate smoothly between -1 and 1, making it suitable for use in machine learning models.

2.4 Conclusion

In this second chapter, we began by exploring the data and getting familiar with it, we then pinpointed the challenges that came with the dataset regarding the lack of quantity and quality data. Afterwards, we proceeded with some preprocessing steps such as value imputation for the missing datapoints and feature scaling, we finally cleaned our data which is now ready to be fed to ML algorithms.

CHAPTER 3

PREDICTIVE MODELING & EVALUATION

3.1 Introduction

This section outlines the process of predictive modeling, including the step of selecting the model after evaluating the performance of all the utilized learning algorithms. We'll also compare the performances after applying the two imputation types seen beforehand.

3.2 Machine Learning model

The dataset is now ready to be used as an input for the ML models, we imputed the missing values, we then normalized the numerical features and finally we encoded the categorical feature (the date), the final dataset look like this:

P1	P2	P3	P4	P5	P6	P7	P8	P9	P10	P11	Objectif	day_sin	day_cos	month_sin	month_cos
0.503001	0.207967	0.183862	0.468750	0.203057	0.708481	0.954008	0.863780	0.959693	0.781170	0.655154	47.966667	0.201299	0.979530	0.5	0.866025
0.436344	0.069563	0.082768	0.729167	0.164483	0.665803	0.944110	0.884271	0.959693	0.819656	0.658557	48.733333	0.394356	0.918958	0.5	0.866025
0.503767	0.206855	0.193330	0.697917	0.298399	0.606381	0.938951	0.905945	0.959693	0.821767	0.638850	46.674746	0.571268	0.820763	0.5	0.866025
0.437094	0.198267	0.161514	0.781250	0.150655	0.675619	0.929397	0.855181	0.959693	0.810433	0.660185	46.723741	0.724793	0.688967	0.5	0.866025
0.539396	0.232374	0.221204	0.802083	0.333333	0.620459	0.932072	0.878492	0.959693	0.781807	0.655273	46.723741	0.848644	0.528964	0.5	0.866025
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
0.491800	0.174728	0.176221	0.610417	0.486754	0.456317	0.959358	0.831594	0.973128	0.864527	0.662037	46.179262	0.299363	-0.954139	0.5	0.866025
0.550919	0.196331	0.176399	0.620833	0.328675	0.655469	0.957223	0.853755	0.383877	0.868567	0.667668	46.179262	0.101168	-0.994869	0.5	0.866025
0.496115	0.175506	0.151074	0.598958	0.393304	0.558181	0.965990	0.874671	0.578503	0.886316	0.662550	46.179262	-0.101168	-0.994869	0.5	0.866025
0.488927	0.269239	0.226933	0.600000	0.287627	0.620960	0.951428	0.861397	0.770441	0.875285	0.660742	46.179262	-0.299363	-0.954139	0.5	0.866025
0.529165	0.178605	0.160172	0.611659	0.377174	0.600054	0.961706	0.858655	0.761733	0.872898	0.600154	46.226514	-0.485302	-0.874347	0.5	0.866025

Figure 3.1: Preprocessed dataset

The next step would be to use multiple off-the-shelf ML algorithms and then compare their

respective performances so we can choose the better one. These are the used algorithms:

- Linear Regression
- Ridge Regression
- Lasso Regression
- ElasticNet Regression
- SVR
- Decision Tree Regression
- Random Forest Regression
- KNN Regression
- Gradient Boosting Regression
- XGBoost Regression
- LGBM

We will employ an 80%-20% train-test split to maximize our utilization of training examples, considering the limited number of data points available, the metrics used for the model comparison are MSE (Mean Squared Error) and the R2 Score. We won't be using the year feature for the training as most of the data was measured in 2020 thus making this feature not useful and it may cause some overfitting.

3.3 Mean Squared Error (MSE)

Mean squared error (MSE) is a widely used metric for evaluating the performance of regression models, it measures the amount of error in statistical models by assessing the average squared difference between the observed and predicted values. In essence, MSE measures how far off the model's predictions are from the true values. A lower MSE indicates that the model's predictions are closer to the actual values, reflecting a better fit to the data. When a model is "perfect" or has no error, the MSE equals zero. As model error increases, its value increases.

$$MSE = \frac{\sum(y_i - \hat{y}_i)^2}{n}$$

Figure 3.2: MSE formula where y-i is the i-th observation, yhat-i is the corresponding predicted value and n the number of observations

3.4 R2 Score

The R2 score, also known as R-squared or coefficient of determination, is one of the most widely used metrics for regression. This metric is a "normalized" version of the MSE (Mean Squared Error).

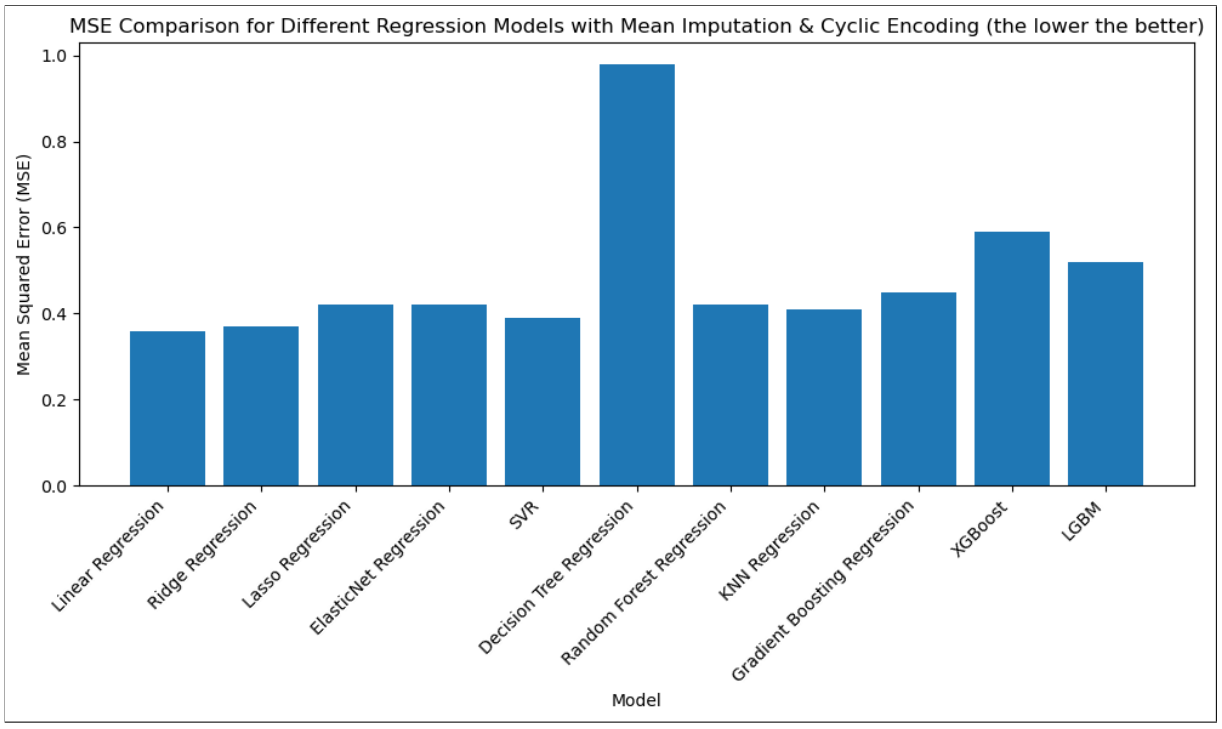
R2 can be seen as the error of the model divided by the error of a basic model that predicts the mean of the variable to be predicted all the time. The higher the model's performance, the higher the R2 score, and is worth a maximum of 100% when all predictions are correct. There is no minimum score, but a simple model predicting the mean value all the time achieves an R2 score of 0%. Consequently, a negative R2 score means that predictions are worse than if the mean value were always predicted.

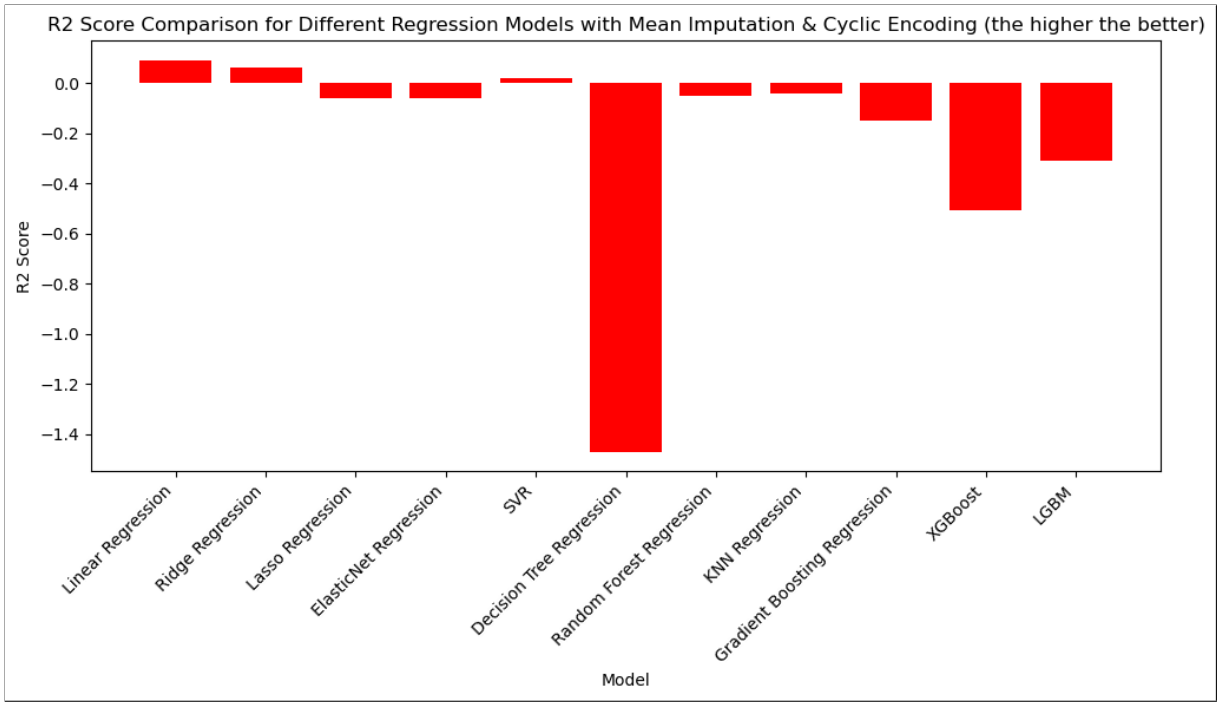
$$R2 = 1 - \frac{\sum_{i=1}^{n} \left(y_i - \hat{y}_i\right)^2}{\sum_{i=1}^{n} \left(y_i - \bar{y}\right)^2}$$

Figure 3.3: R2 formula where y-i is the i-th observed value and yhat-i is the corresponding predicted value, ybar represents the mean of the observations

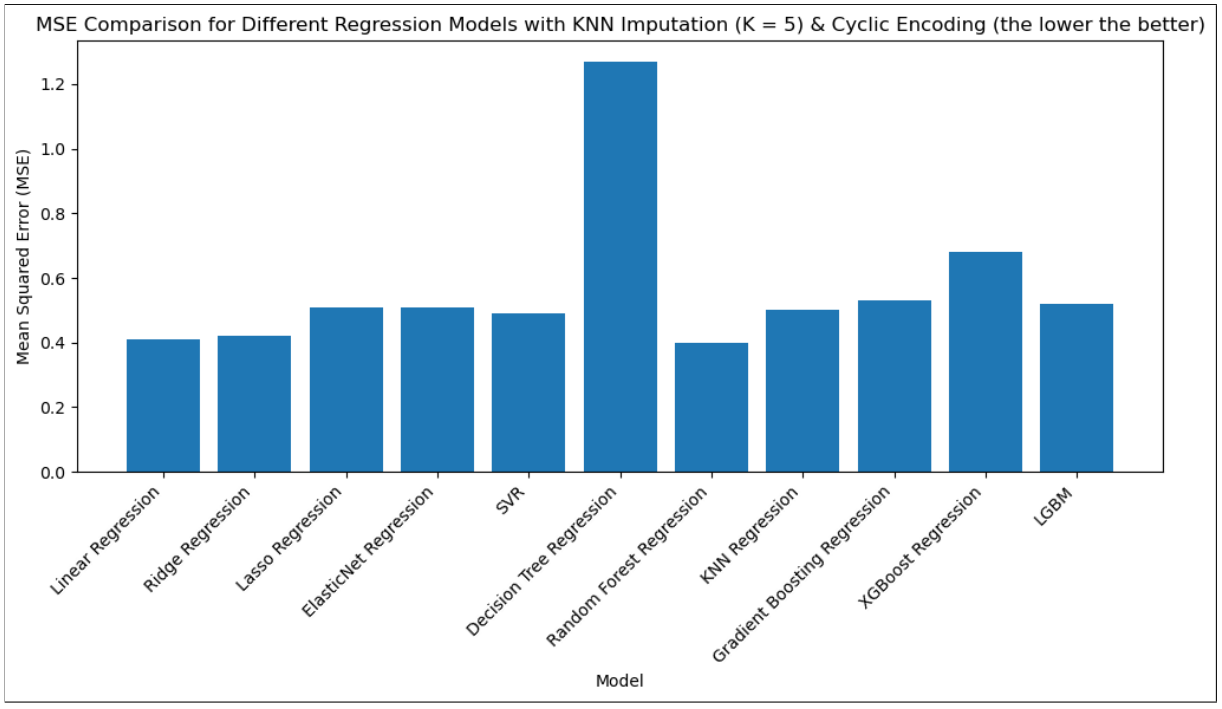
3.5 Experimentations and results

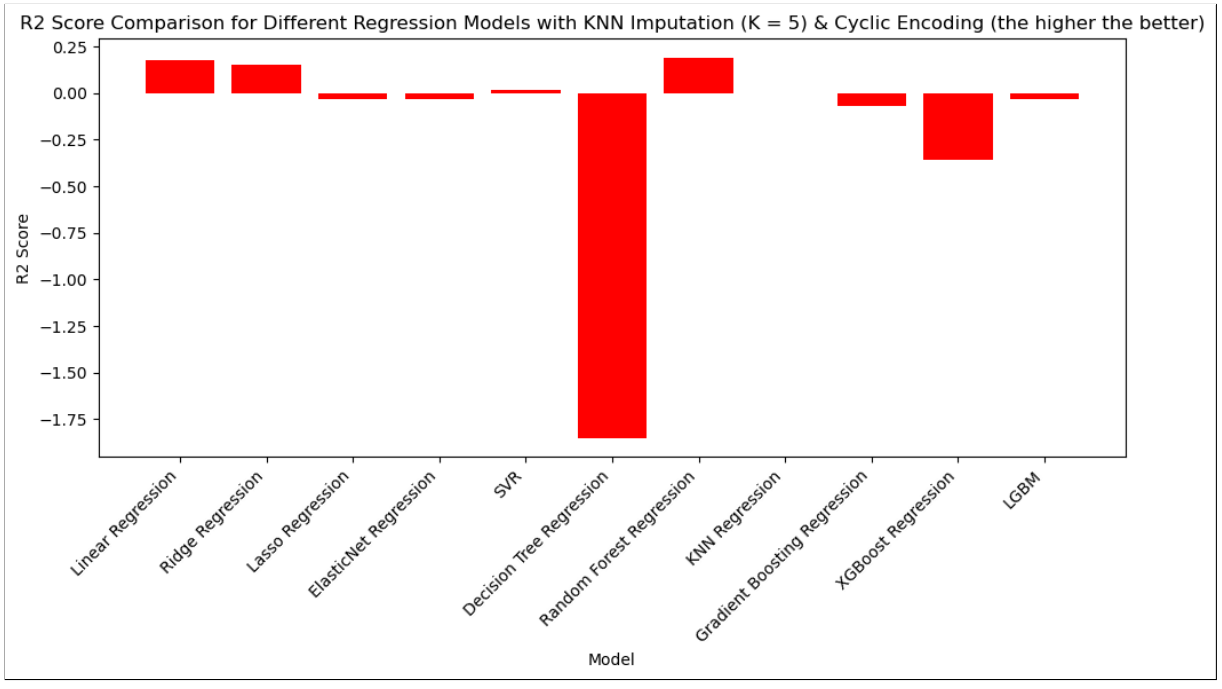
Performances with Mean Imputation





Performances with KNN Imputation





Results

We first notice that overall, the performances are quite bad which can be explained with the small quantity and low quality of the data provided by CIMAT.

- **Mean Imputation:** Best performing model is Linear Regression with MSE=0.36 and R2=0.1%
- **KNN Imputation:** Best performing model is the Random Forest with MSE=0.40 and R2=19% closely followed by Linear Regression with a MSE=0.41 and R2=18%

We'll thus go for a KNN Imputation as it clearly improved all the models used, and for the actual model we chose Random Forest, even though the performances are nearly similar to the Linear Regression ones the former has the edge as it is an Ensemble Learning algorithm and thus is comprised of multiple combined weak learners (in this case Decision Trees) which will generally be better than using only one weak learner.

Random Forest Regression: Random Forest is a versatile and powerful Ensemble Learning algorithm in machine learning. It operates by combining multiple decision trees, each trained on different subsets of the data, to make more accurate predictions. The "random" aspect comes from using random subsets of features and data samples during each tree's training, which helps prevent overfitting and enhances the algorithm's ability to generalize well to new data.

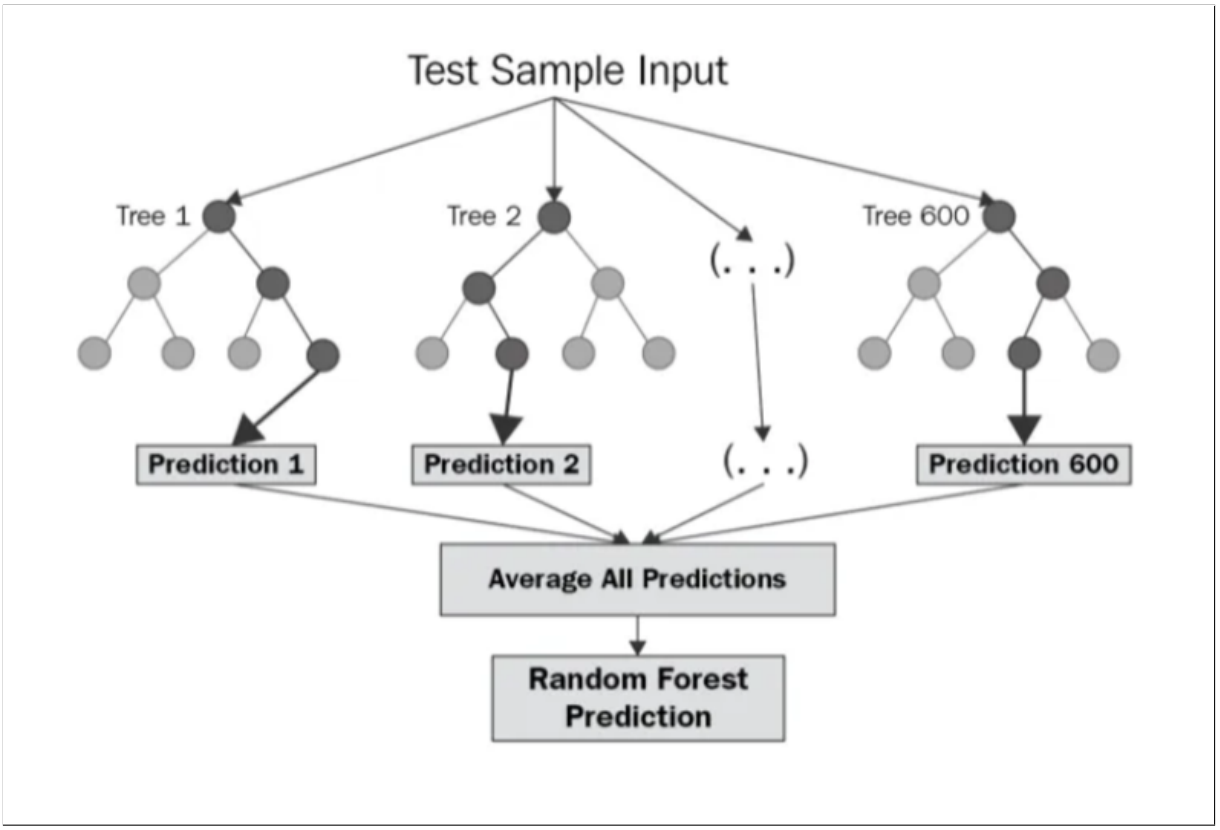


Figure 3.4: Random Forest

The diagram above shows the structure of a Random Forest. You can notice that the trees run in parallel with no interaction amongst them. A Random Forest operates by constructing several

decision trees during training time and outputting the mean of the classes as the prediction of all the trees [3]. To get a better understanding of the Random Forest algorithm, let's walk through the steps:

- Pick at random k data points from the training set.
- Build a decision tree associated to these k data points.
- Choose the number N of trees you want to build and repeat steps 1 and 2.
- For a new data point, make each one of your N-tree trees predict the value of y for the data point in question and assign the new data point to the average across all of the predicted y values.

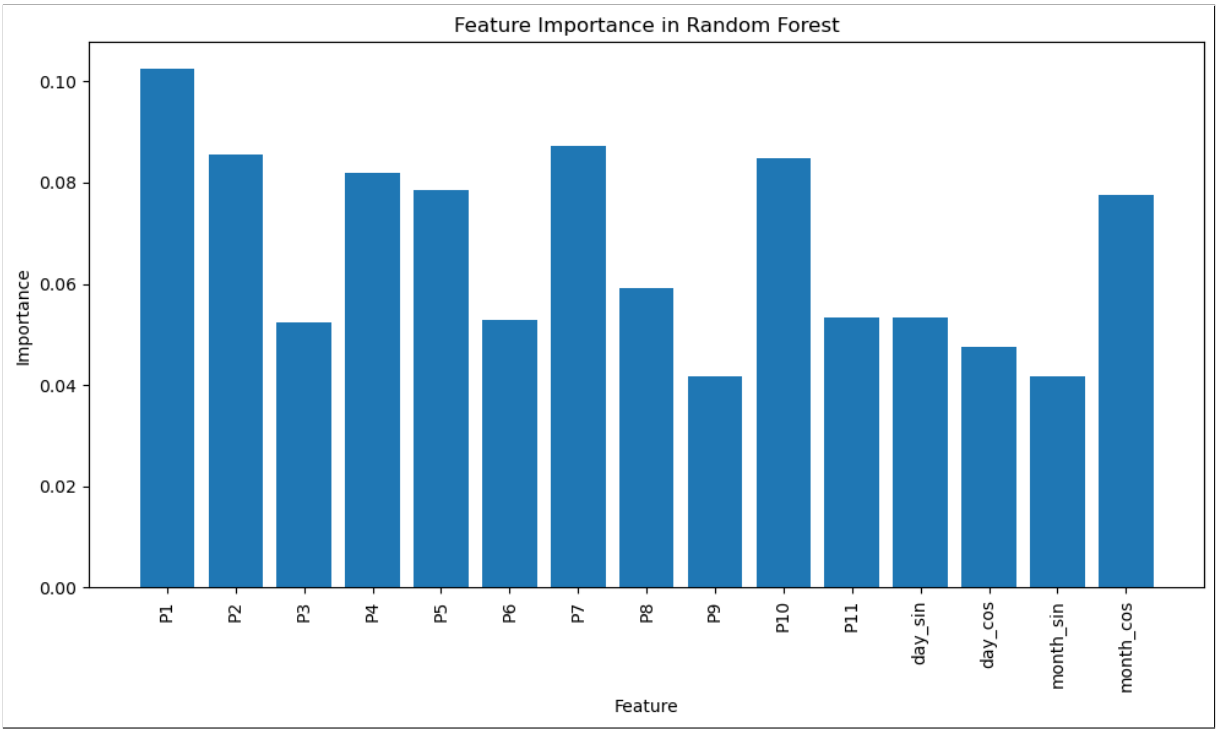


Figure 3.5: Feature's Importance after training

3.6 Conclusion

In this third chapter, we ultimately chose the Random Forest algorithm with a KNN Imputation after making the relevant comparisons. Subsequently, we will proceed to discuss the deployment of our model in a production environment. This aspect will be covered in the upcoming chapter.

CHAPTER 4

MODEL DEPLOYMENT

4.1 Introduction

In this chapter we'll carry out the deployment of our model, we'll begin by creating an API that will take as input new raw and unseen data, this same data will be preprocessed in the same way as the training data and will be given the model's prediction as an output. This API will be deployed into a production environment on the cloud afterwards.

4.2 FastAPI

In the final stages of our machine learning project, we've gone beyond developing the model to create an efficient API using FastAPI. FastAPI is a modern Python framework designed for building APIs quickly and effectively. By integrating our trained model and MinMax scaler, the API can now process new data inputs seamlessly, the same processed data that will be fed to the model in order to get our predictions. To streamline deployment, we've saved both the model and scaler in a single pickle file. We'll test our API using Postman.

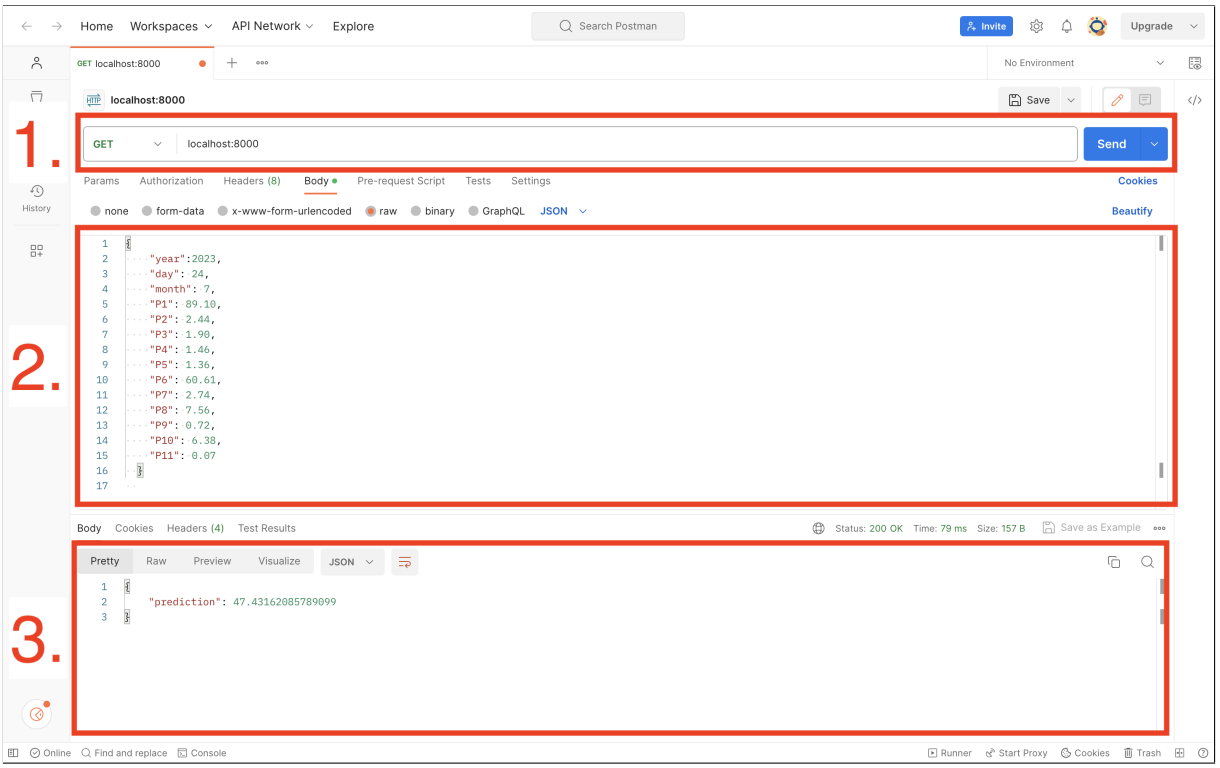


Figure 4.1: Postman Interface

Our API is running on port 8000 which is the default port of Uvicorn; the ASGI server used by FastAPI to serve the APIs.

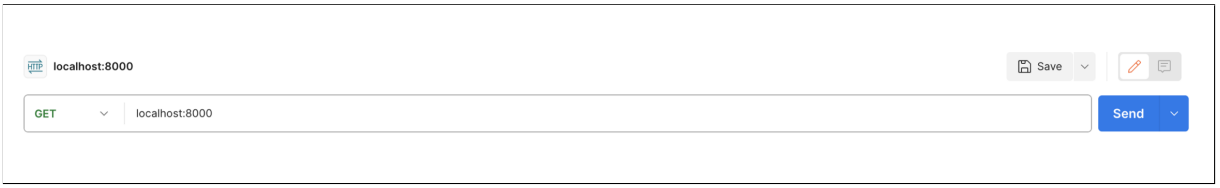


Figure 4.2: **1.** The request URL (or endpoint) in Postman is entered here

The input raw unseen data that will be fed into the API is in the JSON format. The user inputs the date of the measurement and the value of each one of the 11 P sensor, the API will process the data the same way we did earlier with the training data and feed it into the model.

Figure 4.3: **2.** Input data to be fed into the model via our API

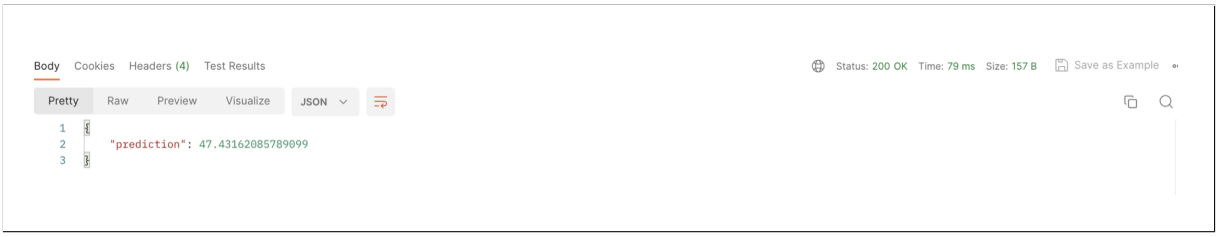


Figure 4.4: **3.** API's output which is our model's prediction for the user's input

4.3 Railway

So far, we have created an API that outputs predictions locally, the next step would be to deploy it into a web application with the help of a cloud service.

To make deployment smoother, we've utilized Railway, a user-friendly platform that simplifies the process of deploying and hosting web applications. With Railway, managing the infrastructure and scalability of our API becomes easier. The combined power of FastAPI and Railway now offers web accessibility for our machine learning model, allowing users to get accurate predictions using their devices. This collaboration between FastAPI, Railway, and our serialized model transforms our project into a practical and deployable application.

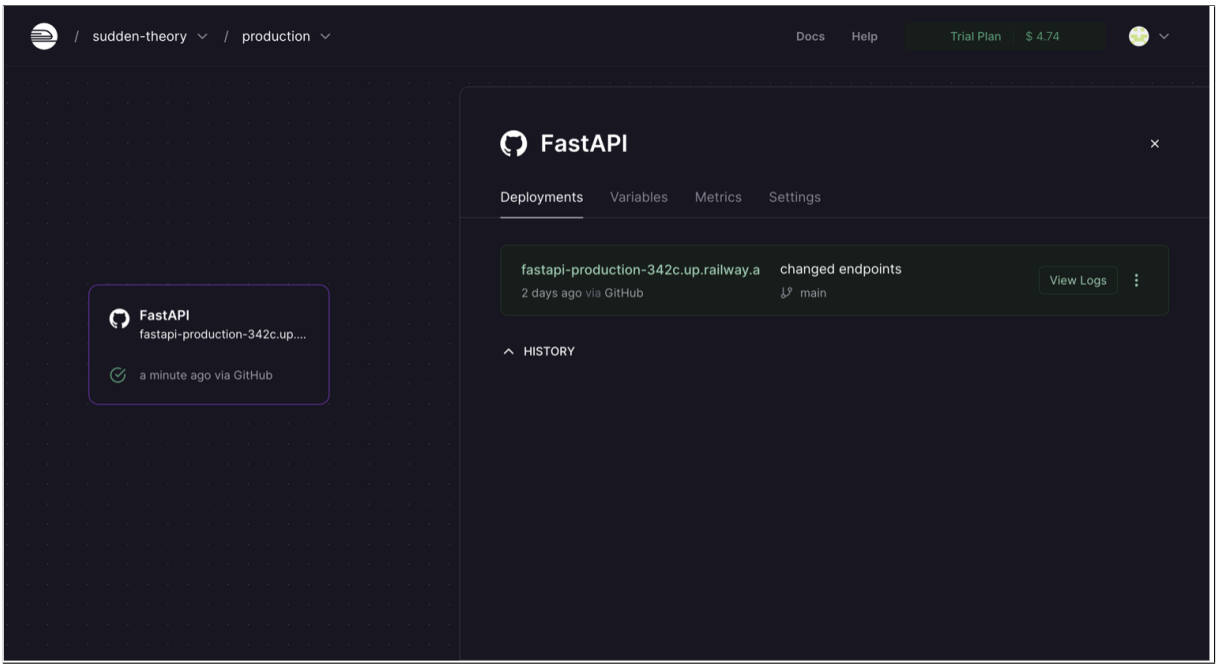


Figure 4.5: Railway's main page

Deployment with Railway is easy, it automatically build and deploy our API using the github repository in which is hosted the said API.

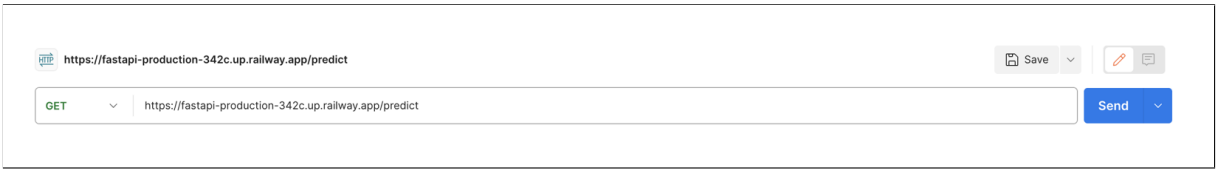


Figure 4.6: New endpoint: we use the web link provided by Railway

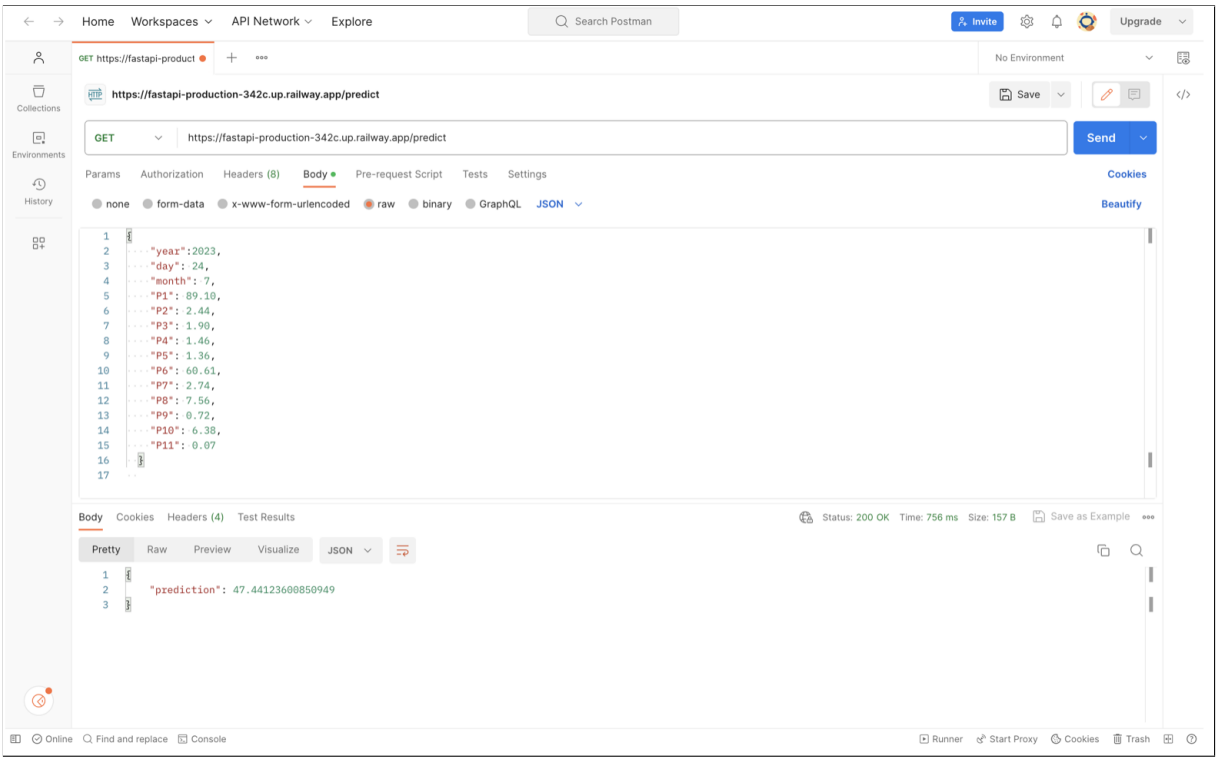


Figure 4.7: With Postman, we test again our API which is now hosted in Railway and we get the same prediction

4.4 Conclusion

This chapter marks the conclusion of our build-from-scratch approach, in which we developed the model and we deployed it by hand using the Python language and the relevant frameworks and libraries.

CHAPTER 5

AUTOML APPROACH

5.1 Introduction:

This chapter is dedicated to the AutoML approach. This phase encompassed the exploration of a renowned AutoML platform – DataRobot. Within this chapter, we provide a concise glimpse of this exploration, highlighting the incorporation of automated methodologies to enhance and simplify the predictive modeling process and the deployment phase.

5.2 Datarobot

DataRobot emerges as a multifaceted solution that encompasses an array of functionalities tailored to various user levels, spanning from novices to experts in the realm of data science. The platform distinguishes itself through a comprehensive suite of tools and automated features, effectively streamlining intricate processes such as data preprocessing, model selection, and optimization of hyperparameters. Noteworthy is its emphasis on accessibility, rendering complex machine learning endeavors more approachable through an intuitive interface and guided workflows. By combining advanced algorithms with user-friendly design, DataRobot encapsulates the democratization of machine learning, enabling a broader spectrum of individuals to participate in the creation, deployment, and management of potent predictive models.

5.2.1 Project's setting

We create a project in Datarobot's main page and we upload our dataset, we'll use the dataset with the target values predicted by our LGBM model back in chapter 2 but we'll leave the features unchanged.

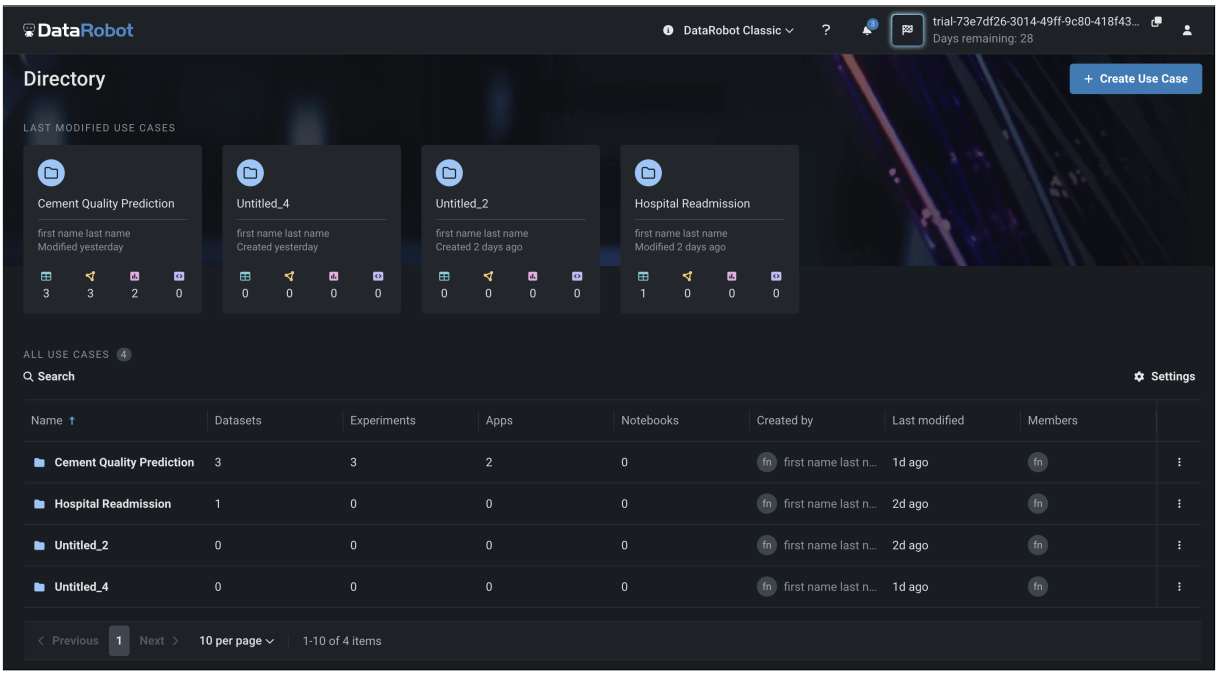


Figure 5.1: Datarobot main page

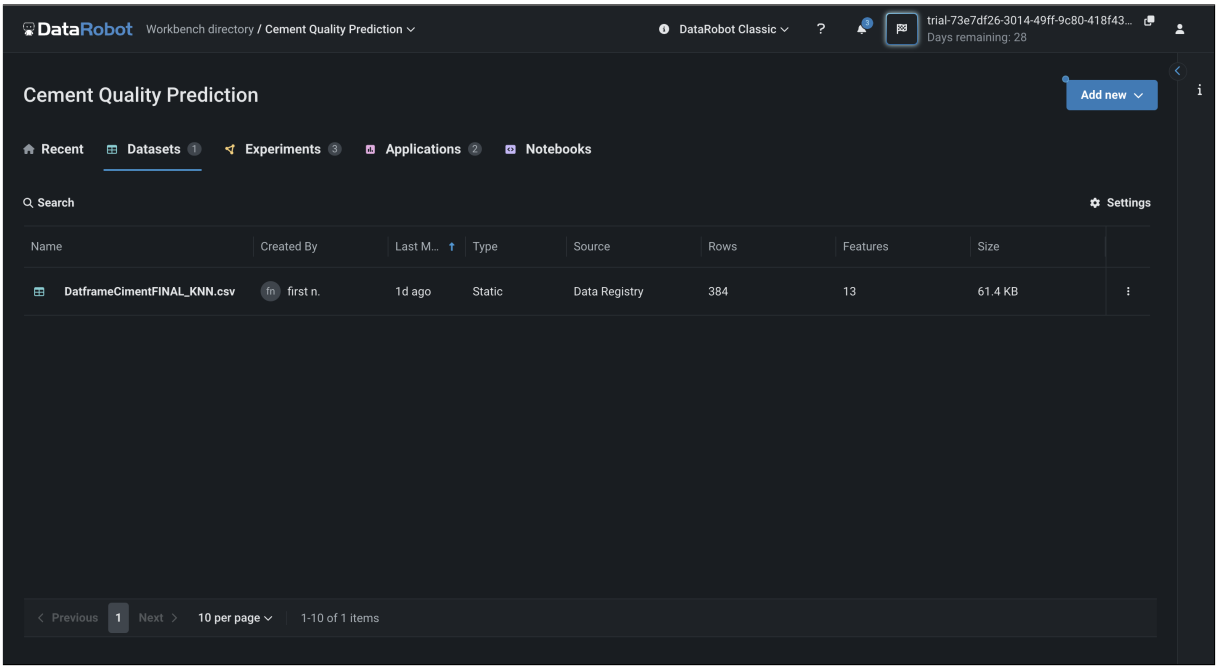


Figure 5.2: Cement Quality Prediction Project created in Datarobot

Our dataset is uploaded

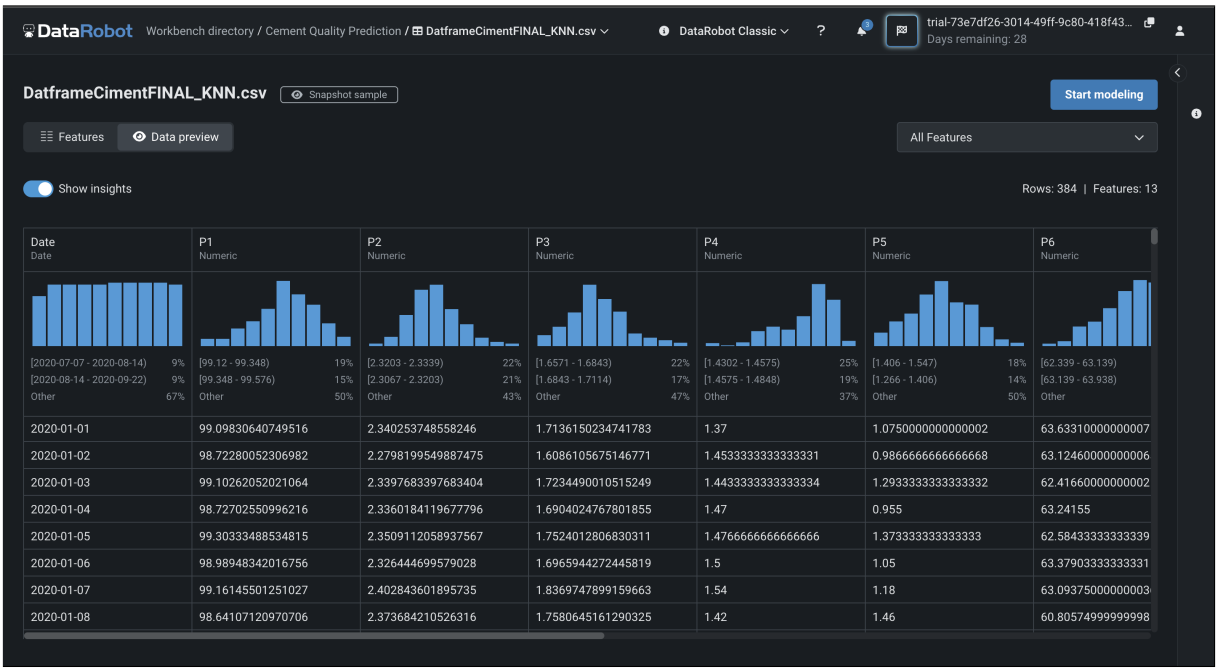


Figure 5.3: Data's insights

Some insights given by the platform about our data before the modelization step.

5.2.2 Predictive Modeling

After we upload our dataset, we click on the "start modeling" and we begin the Datarobot modelization part of the project, we choose our target function which is the "Objectif" column and then Datarobot will start building multiple models.

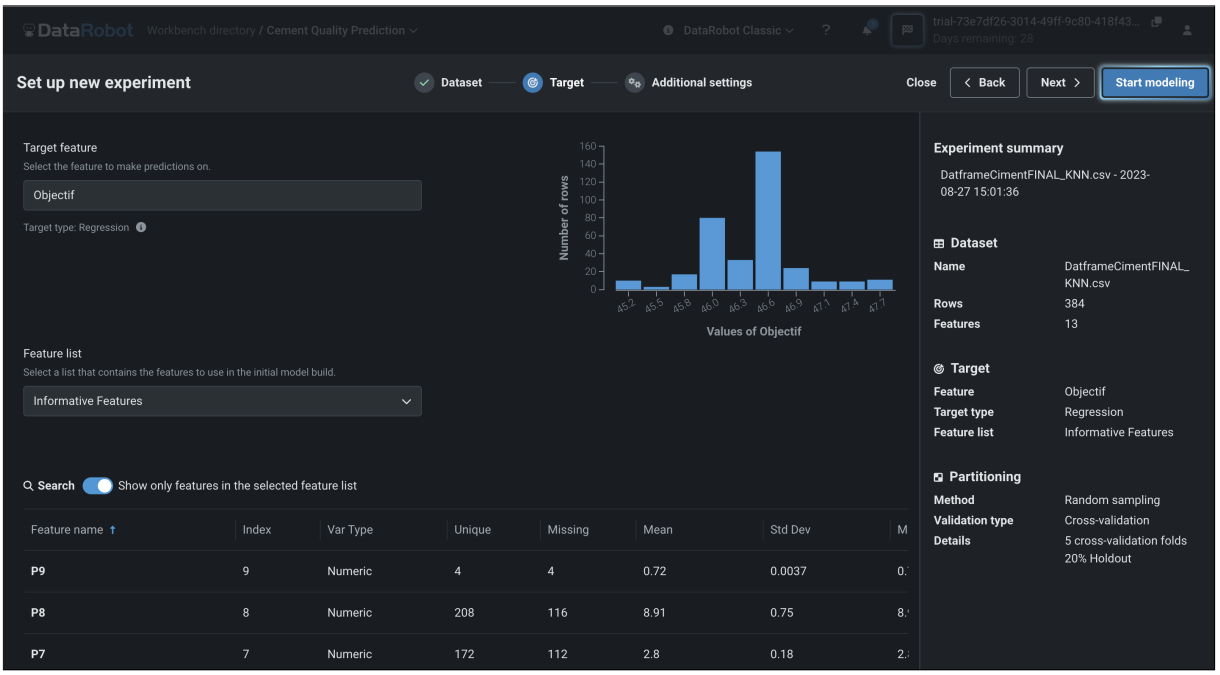


Figure 5.4: Target choice for the modelization step

Datarobot has built multiple models, the same models are ranked from best to worst performer. We'll base our choice on the R2 score metric out of the numerous metrics provided by Datarobot. We'll then chose the Random Forest algorithm that has a R2 score of 18% which is the best performer. We can notice that the performances and model choice are similar to the previous 'from scratch' approach.

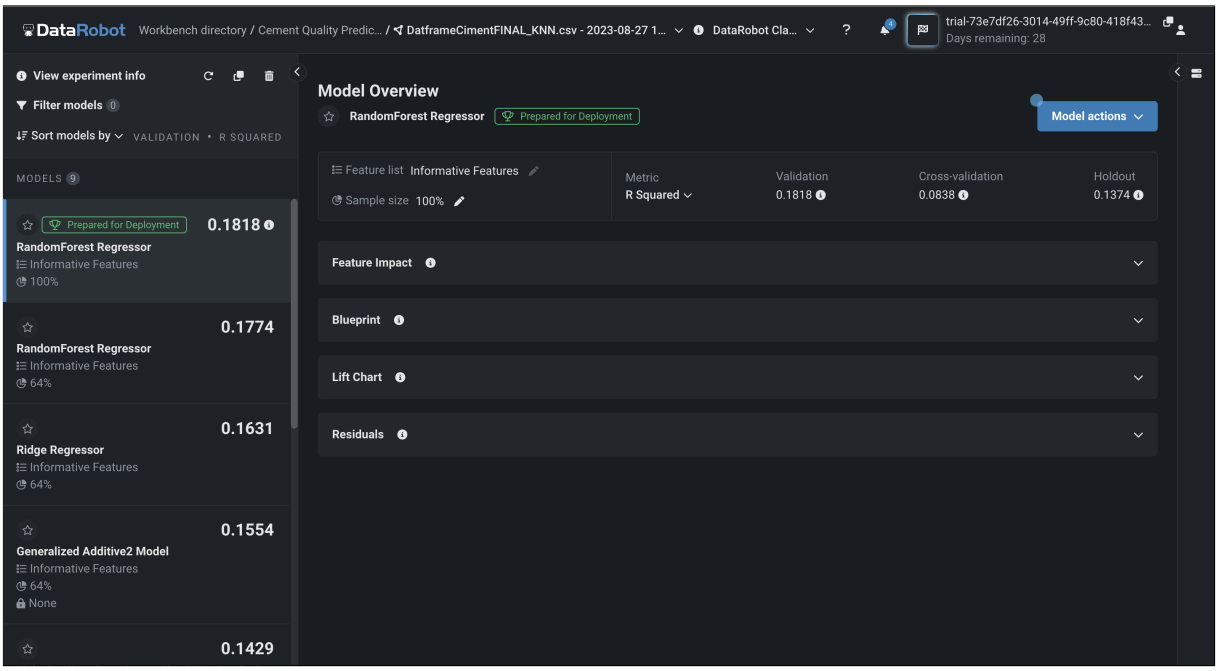


Figure 5.5: Models built by Datarobot

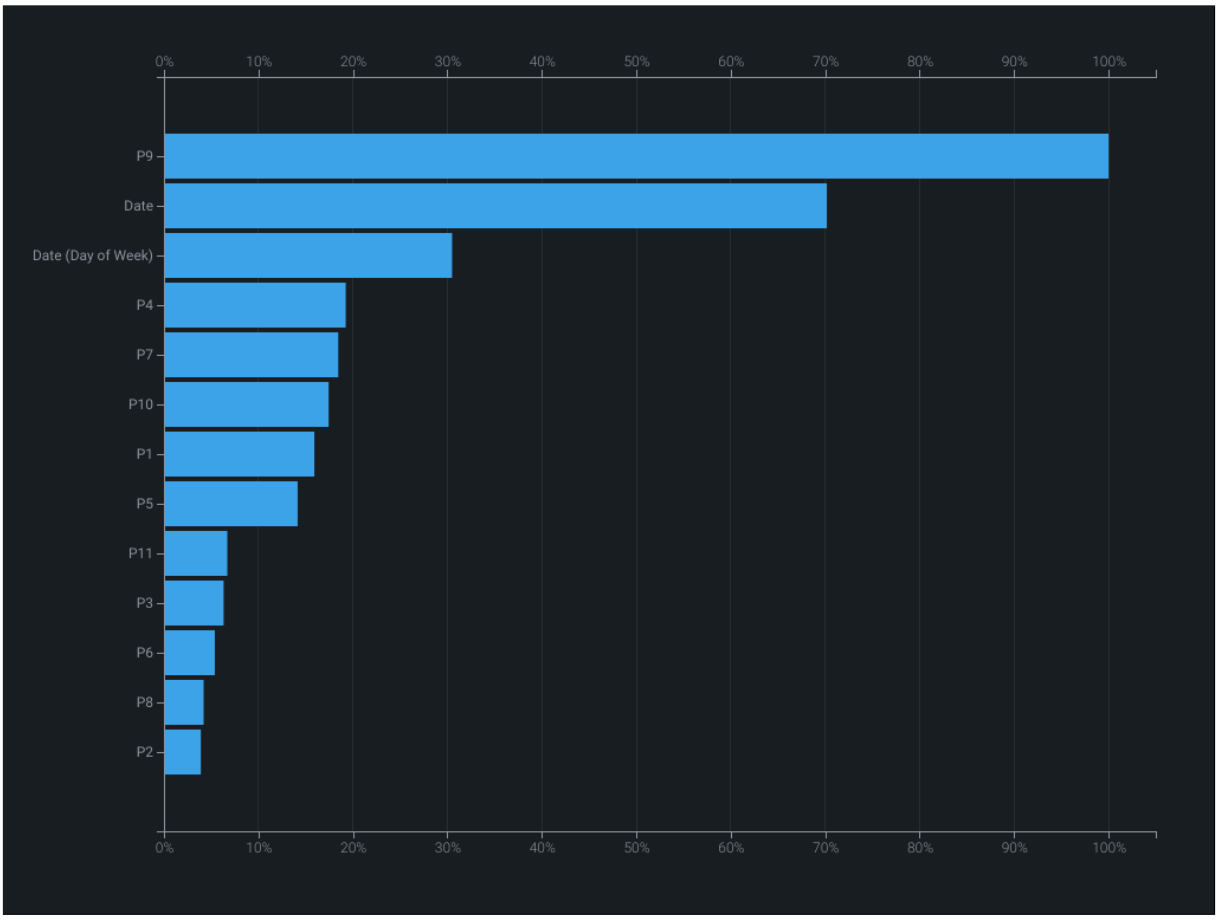


Figure 5.6: Feature Importance for Random Forest Regressor in Datarobot

5.2.3 Preprocessing steps

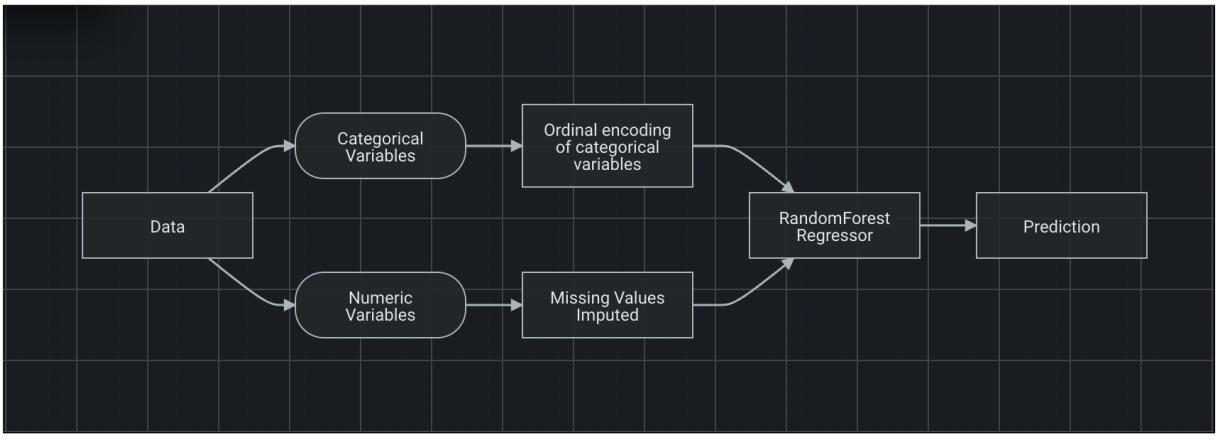


Figure 5.7: Automated preprocessing steps done by Datarobot

We can see the workflow done by the platform when cleaning the data:

- **Handling of date column:** Datarobot used the Ordinal Encoding when handling the date column. Ordinal encoding for a date column involves transforming date values into numerical representations that retain the sequential and chronological order of the dates. The encoding assigns a unique number to each date based on its position in the chronological sequence. For instance, for values like "2020-08-27," "2020-08-28," "2020-08-29," and so on, ordinal encoding would map these dates to consecutive numbers like 1, 2, 3, etc., according to their order in time.
- **Imputation for missing values:** Datarobot used the Median Imputer, which is basically replacing the missing values with the median.
- We notice that Datarobot didn't use a scaler to normalize the data.

5.2.4 Model Deployment

After choosing our model, we will deploy it using Datarobot. DataRobot integrates with popular external cloud providers, such as Amazon Web Services (AWS), Microsoft Azure, and Google Cloud Platform (GCP), to deploy and host machine learning models, the deployment location is managed by DataRobot's backend infrastructure, and we don't have to select a specific cloud provider or server ourselves. DataRobot's deployment environment is abstracted away from the user interface to simplify the deployment process.

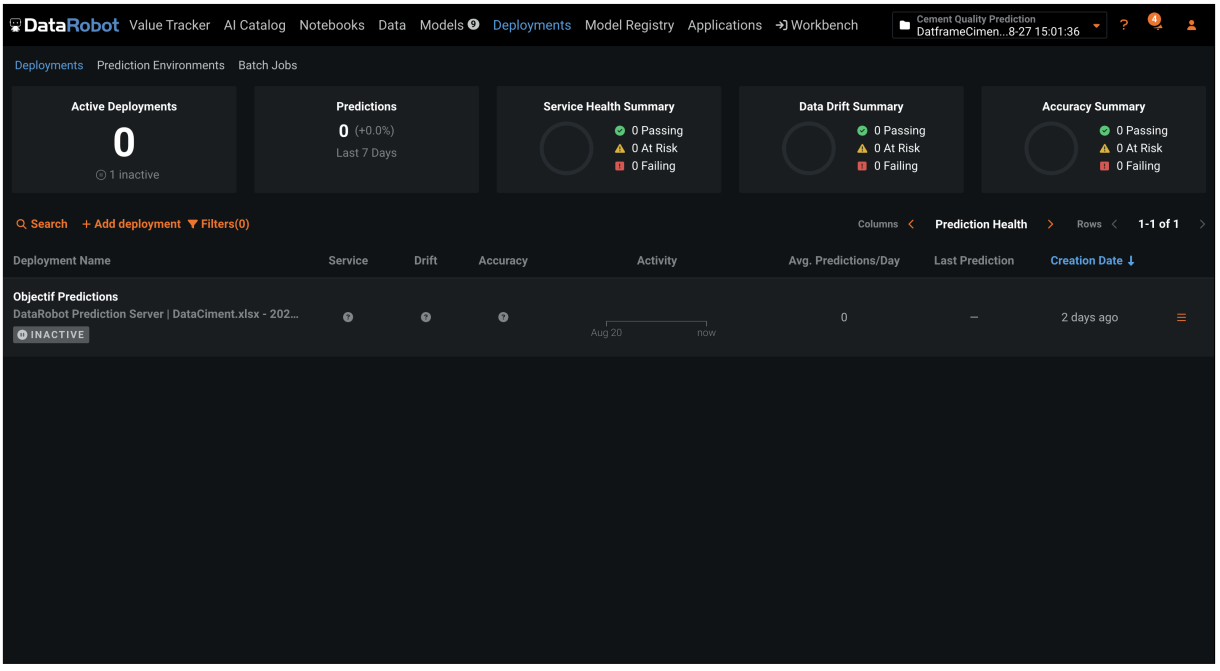


Figure 5.8: Datarobot's deployment environment

We select our Random Forest Regressor to be deployed into Datarobot's deployment environment.

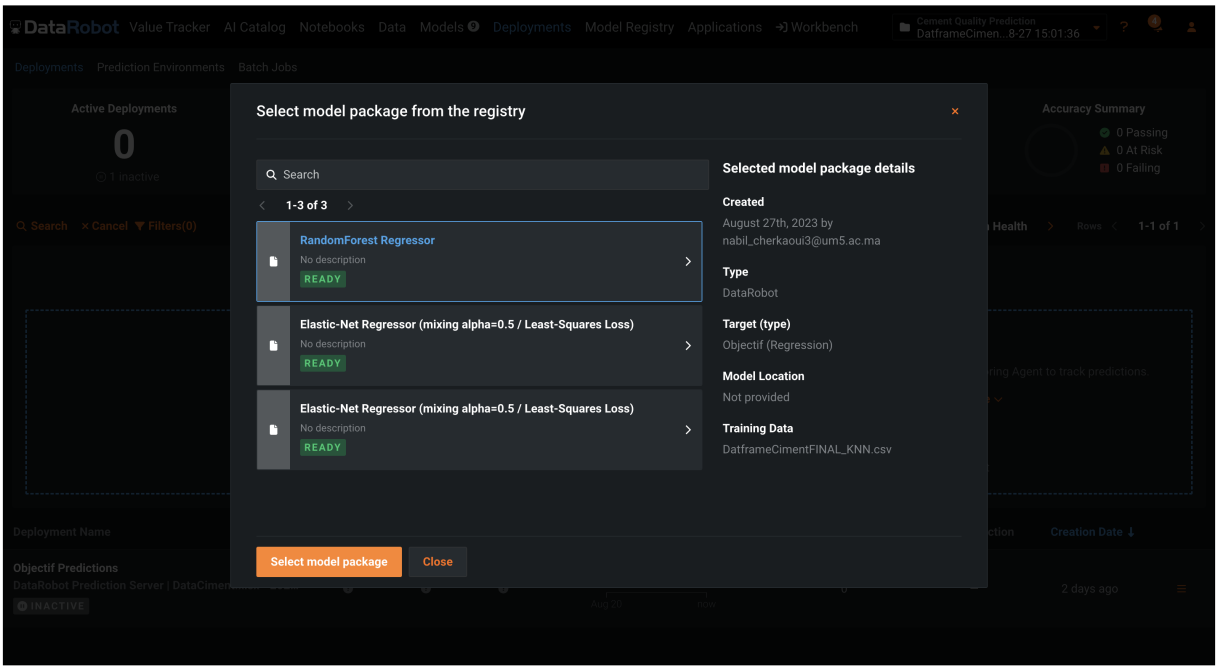


Figure 5.9: Model selection for the deployment

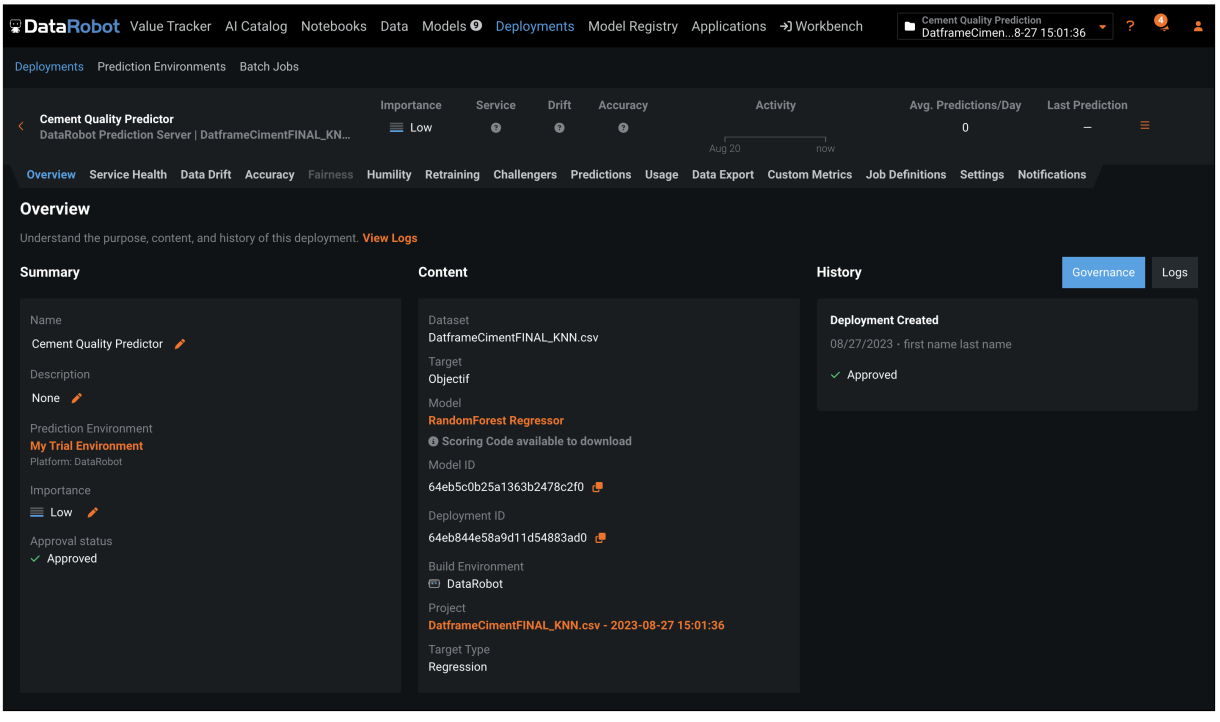


Figure 5.10: Model deployed in Datarobot

App creation: Our model is deployed successfully. Now to make new predictions using our deployed model, we need to create an application that will connect to the deployed model

endpoint. Datarobot is providing us with a predictor application template to facilitate the process.

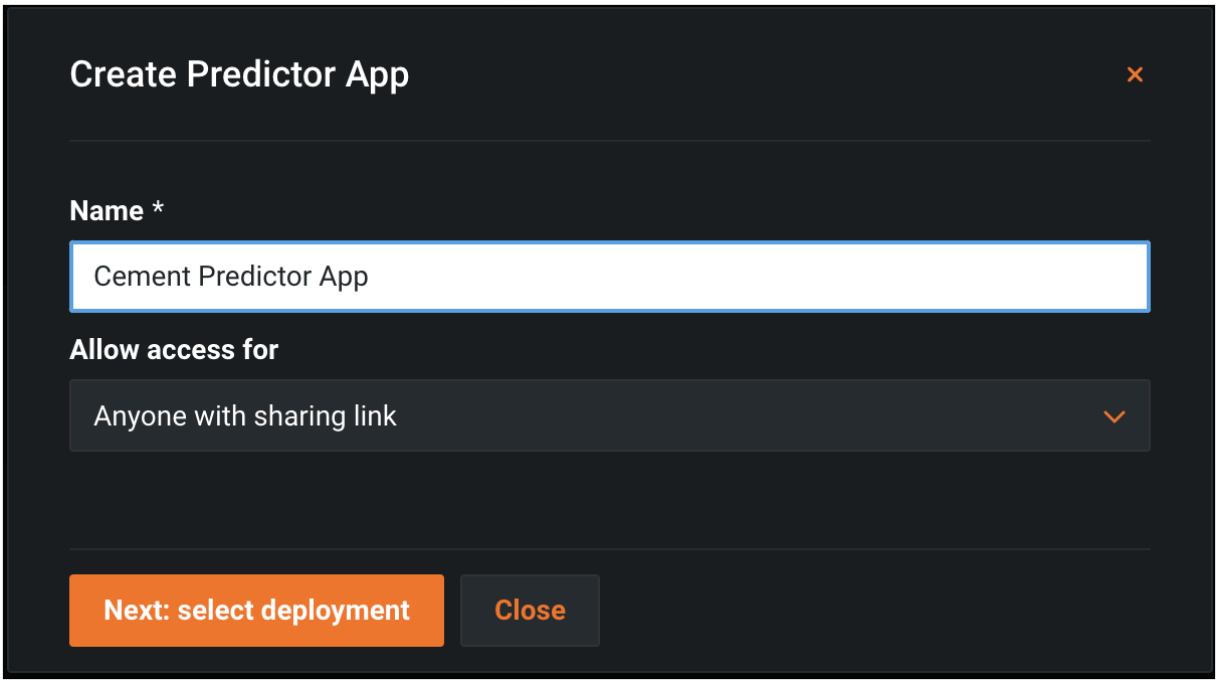


Figure 5.11: Predictor application template in Datarobot

We select our deployed model to be used for our application.

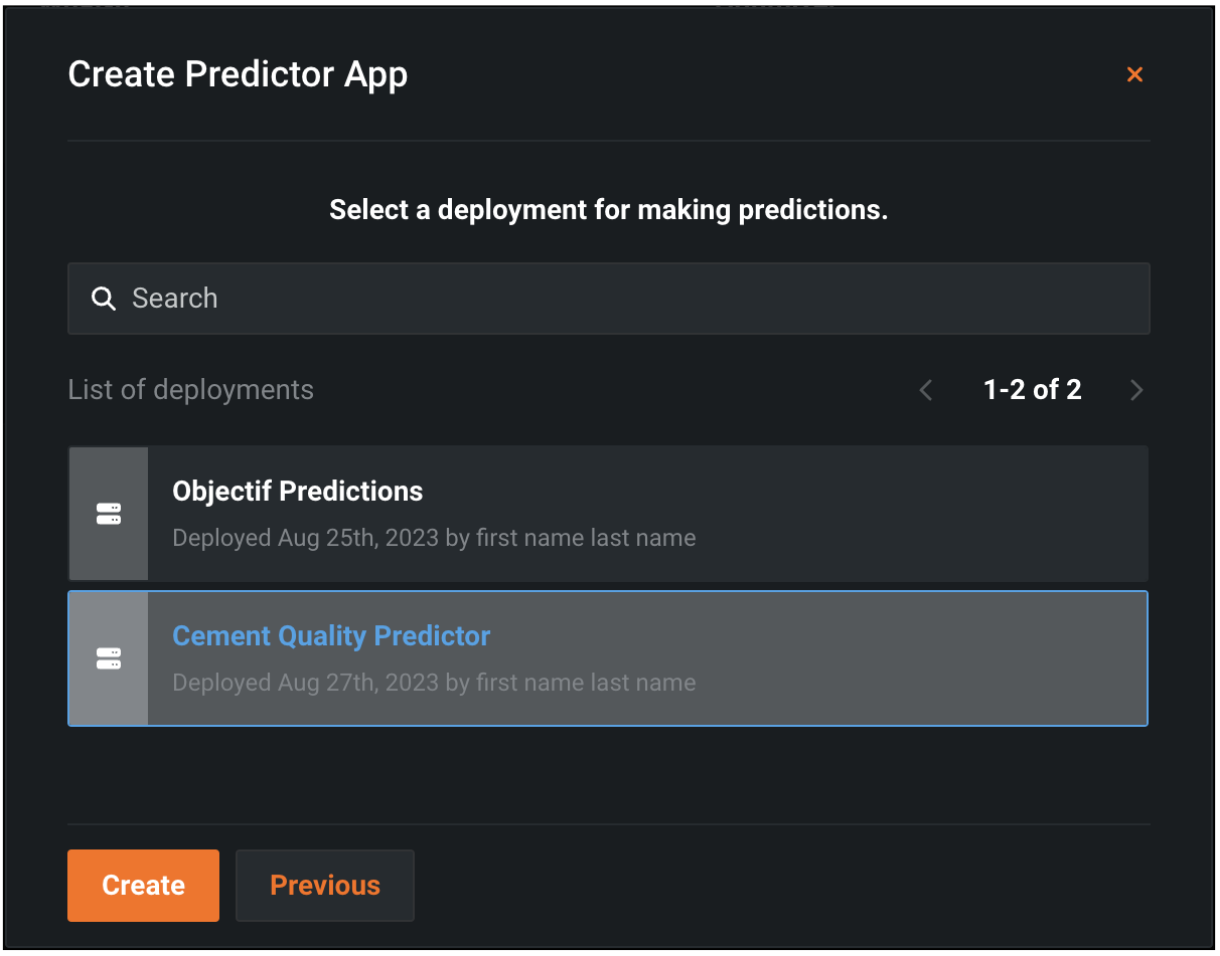


Figure 5.12: Deployed model selection for the application

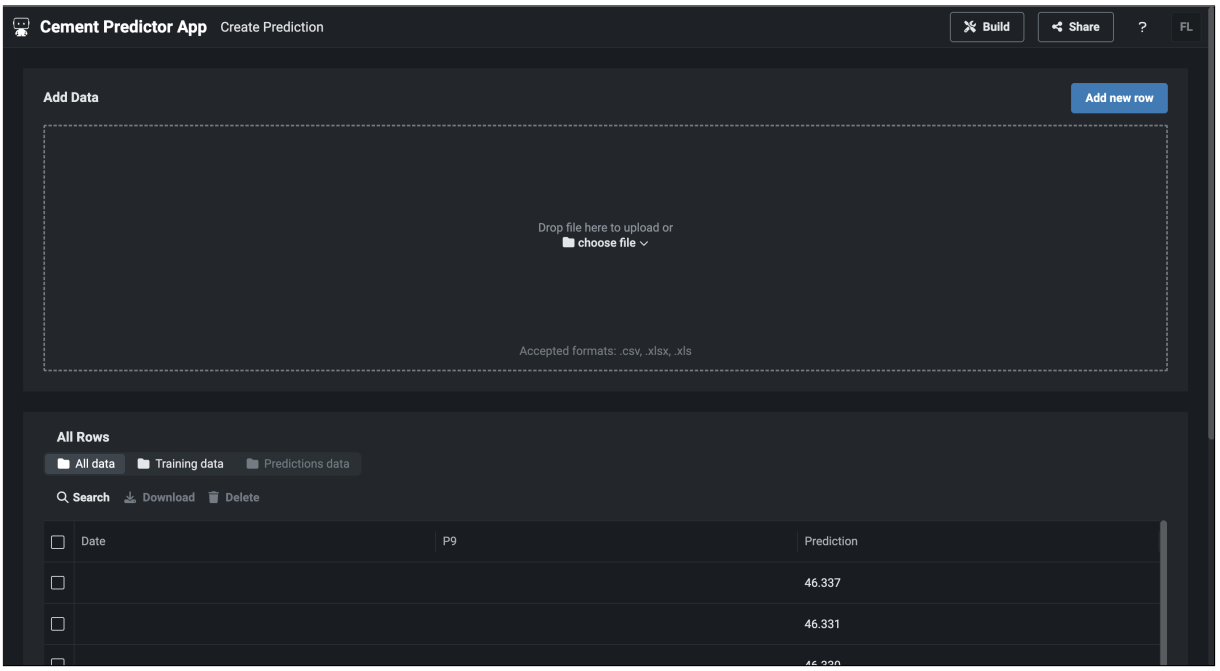


Figure 5.13: The Cement Predictor Application home page

Our app is now well and running, we'll go to the "Create Prediction" tab to create a prediction from new unseen feature data.

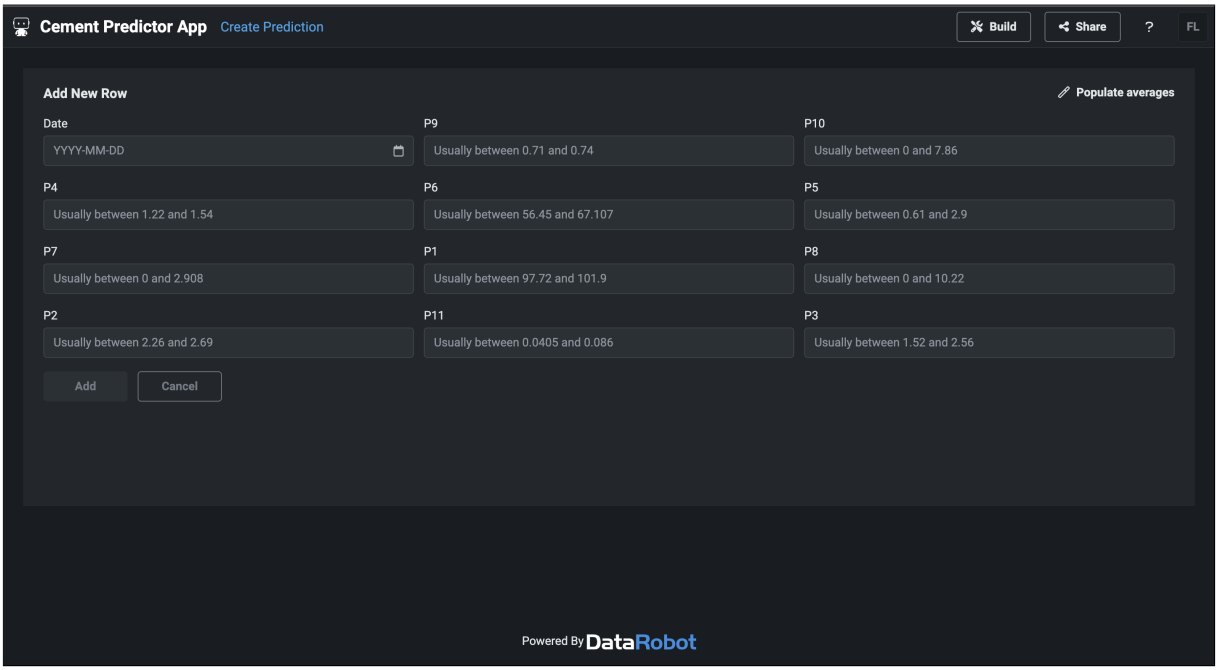


Figure 5.14: New input feature data form for the application

In the form shown above we enter the new unseen feature data, this data will be fed into

the model in order to get the new prediction through our application.

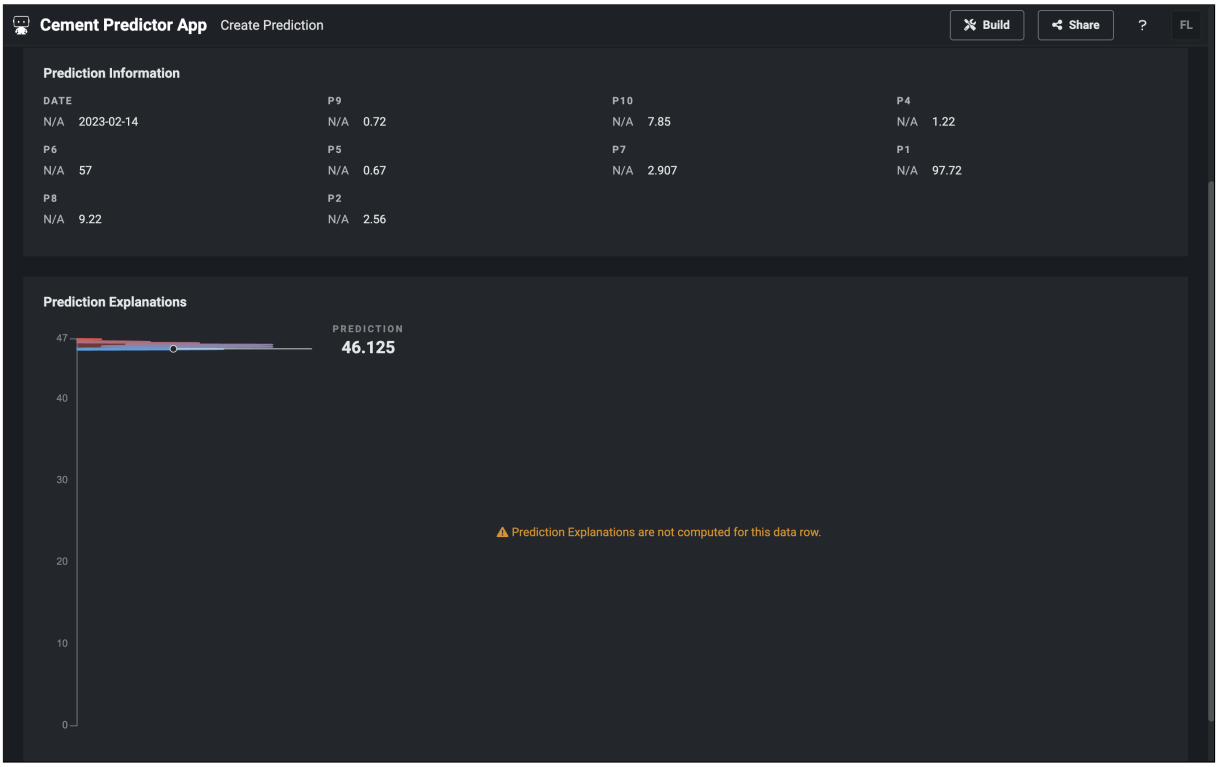


Figure 5.15: The predictor app's output

The predictor app successfully gives us a prediction based on the unseen inputed data.

5.3 Conclusion

As a conclusion, in this chapter we utilized the DataRobot AutoML tool in order to achieve the required goals of this project. From creating a ML model to deploying it, everything can be done in one platform such as Datarobot. We can clearly point out the advantages of this new way of ML development which are, among others, the rapidity and streamlining of the workflow and also the democratisation of the field. On the other hand, such methods are less customisable so we don't have the same level of control on each step as we do on a more classical approach. Finally, in terms of performance, we notice that in our case both techniques resulted into models that are performing similarly.

CHAPTER 6

TOOLS

6.1 Introduction

In this upcoming chapter, we delve into an exploration of the array of tools that have played a pivotal role throughout the duration of the internship. This segment unveils an in-depth portrayal of the various tools harnessed during the course of the internship's journey.

6.2 Python

Figure 6.1: Python logo

Python is a programming language that holds a vital role in the realm of Data Science, smoothly blending adaptability and efficiency into analytical tasks. Its significance in data science emerges from its versatile nature, handling various tasks from data preprocessing to complex model

creation. Python's powerful ecosystem, including libraries like NumPy, pandas, and scikit-learn, empowers data scientists to manage complex data, conduct advanced statistics, and build intricate machine learning models. This blend of Python's user-friendly structure and diverse data-focused libraries places it at the forefront, allowing data scientists to extract insights, drive innovation, and shape transformative results in the field of data science.

6.3 Jupyter

Figure 6.2: Jupyter logo

Jupyter Notebooks, a recognized tool in interactive computing, effectively combines versatility and user-friendliness in the context of code documentation and execution. Its significance arises from its inherent ability to seamlessly integrate code, visual elements, and explanatory text within a single environment. Supported by interactive features and extensions, Jupyter Notebooks empower a diverse user base including data scientists, researchers, and educators to conduct exploratory analyses, present results, and facilitate collaboration. This combination of Jupyter's accessible interface and interactive capabilities positions it as a valuable resource, enabling users to analyze data, share insights, and engage in code-driven narratives with convenience.

6.4 Pandas

Figure 6.3: Pandas logo

Pandas, a recognized tool in data manipulation, adeptly combines functionality and user-friendliness for data analysis and manipulation tasks. Its significance lies in its inherent ability to effectively handle a wide range of tasks, from data loading and cleaning to transformation and analysis. Supported by a diverse range of functions and methods, Pandas empowers data practitioners to work with structured data, conduct complex operations, and draw insights efficiently. This combination of Pandas' accessible syntax and its comprehensive suite of data-focused capabilities establishes it as a valuable asset, enabling users to work with data, perform analytical tasks, and extract meaningful insights in a straightforward manner.

6.5 Scikit-learn

Figure 6.4: Scikit-learn logo

Scikit-learn stands as a cornerstone in the world of data science, infusing adaptability and efficiency into analytical endeavors. Its significance springs from its inherent flexibility, deftly handling tasks from constructing predictive models to intricate analyses. With an extensive array of algorithms and tools, scikit-learn equips data scientists to navigate complex data landscapes, perform advanced statistical computations, and construct sophisticated machine learning models. This fusion of scikit-learn's user-friendly design and its robust suite of data-centric capabilities establishes it as an essential asset, empowering data scientists to uncover insights, nurture inventive solutions, and mold transformative outcomes in the realm of data science.

6.6 Seaborn

Figure 6.5: Seaborn logo

Seaborn, a prominent contender in data visualization, adeptly blends style and functionality in the realm of graphical representation. Its significance stems from its inherent capacity to handle a range of visual tasks, from generating statistical plots to depicting intricate data patterns. Supported by a versatile array of customizable options, Seaborn equips data analysts to navigate complex datasets, highlight trends, and convey information effectively through visualizations. This convergence of Seaborn's user-friendly design and its rich set of visualization tools positions it as a valuable resource, allowing analysts to uncover insights, communicate findings, and craft informative data stories.

6.7 FastAPI

Figure 6.6: FastAPI logo

FastAPI, a notable contender in web application development, skillfully combines speed and functionality to streamline the process of creating APIs. Its significance lies in its capacity to

effectively handle tasks spanning from routing HTTP requests to data validation. Supported by a versatile set of features and tools, FastAPI empowers developers to construct reliable APIs, manage data effectively, and facilitate seamless communication between clients and servers. This synergy between FastAPI's efficient syntax and its comprehensive range of API-oriented capabilities positions it as a valuable tool, enabling developers to create responsive, efficient, and scalable web APIs with convenience.

6.8 Railway

Figure 6.7: Railway logo

Railway, a recognized platform in web hosting and deployment, effectively combines reliability and ease of use for hosting web applications and APIs. Its significance lies in its inherent ability to simplify the deployment, management, and scaling of applications. Supported by a variety of tools and features, Railway empowers developers to concentrate on their application development without the intricacies of infrastructure management. This combination of Railway's user-friendly interface and deployment-focused capabilities establishes it as a practical tool, enabling users to deploy and manage web projects conveniently and effectively.

6.9 Datarobot

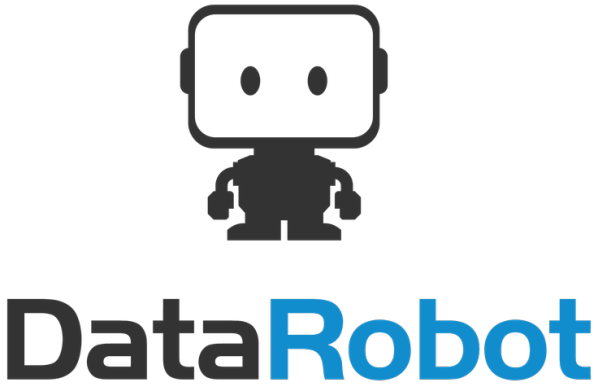


Figure 6.8: Datarobot logo

DataRobot, a recognized player in automated machine learning, effectively combines innovation and efficiency in the realm of predictive model building. Its significance arises from its inherent ability to streamline the model creation process by automating tasks ranging from data preprocessing to algorithm selection. Supported by a diverse range of algorithms and features, DataRobot empowers data professionals to expedite the model development cycle and derive insights more efficiently. This combination of DataRobot's automated approach and its comprehensive suite of machine learning capabilities establishes it as a practical tool, allowing users to enhance the model-building process and attain predictive insights with convenience.

6.10 Conclusion

In conclusion, the culmination of this chapter unveils a comprehensive overview of the tools that have served as the bedrock of the internship's endeavors. The diversity and synergy of these tools have collectively contributed to the successful execution of tasks and the attainment of key milestones

GENERAL CONCLUSION

The objective of this internship was to develop a machine learning model capable of determining, through its predictions, whether certain raw materials should be included in the cement production process at CIMAT company. Unfortunately, our efforts did not result in a robust model with optimal performance, largely attributed to the limitations of the training data available. Despite this, we successfully created a fully operational model, which was subsequently deployed into a production environment. The remaining challenge lies in the model's performance, which was hindered by the data constraints. With the inclusion of more comprehensive training data, there is potential to enhance the model's performance and achieve improved outcomes.

During our internship, we pursued two distinct approaches. The first, known as the "build-from-scratch" approach, involved constructing and deploying a model entirely from the ground up. This methodology leveraged the Python programming language along with pertinent frameworks, including Pandas and Scikit-learn for data preprocessing and modeling, respectively. Additionally, FastAPI was employed to craft the API, which was subsequently deployed via Railway.

Another avenue we explored was the AutoML approach. In this context, we harnessed the prowess of the DataRobot platform, a frontrunner in this rapidly growing domain. DataRobot, a pioneering platform, automates the entire spectrum of ML development - from initial model construction to eventual deployment. This automation streamlines the process, rendering it accessible to individuals who are not necessarily ML experts and thus contributing to the democratization of the field.

In the "construct-from-scratch" approach, we embraced familiar tools such as Jupyter Notebooks, Python frameworks like Pandas, Scikit-learn, FastAPI and more. This route granted us fine-grained control over every step, enabling meticulous customization and a deep understanding of the process. However, it also demanded extensive manual effort, potentially lengthening the development cycle. In contrast, the AutoML journey took us through an innovative path. Utilizing the DataRobot platform, we encountered an automated landscape where ML development, right from inception to deployment, was streamlined. The benefits were evident: rapidity,

accessibility for non-experts, and efficient resource utilization. Yet, the trade-off came in terms of customization flexibility and a less hands-on involvement. Balancing these two approaches was a testament to their distinct advantages, enabling us to navigate both the precision of classical techniques and the agility of automated processes.

Engaging in this project has been a remarkable avenue for both personal and professional growth. The experience garnered has been invaluable, serving as a platform to enhance our comprehension and expertise in the realms of data science and machine learning. This journey has proved immensely beneficial, affording us the opportunity to refine our skills and deepen our understanding in these domains.

REFERENCES

[1]: https://www.techtarget.com/searchenterpriseai/definition/data-science

[2]: Nichols JA, Herbert Chan HW, Baker MAB. Machine learning: applications of artificial intelligence to imaging and diagnosis. Biophys Rev. 2019 Feb;11(1):111-118. doi: 10.1007/s12551-018-0449-9. Epub 2018 Sep 4. PMID: 30182201; PMCID: PMC6381354.

[3]: https://levelup.gitconnected.com/random-forest-regression-209c0f354c84